

机器学习

目录

1	人工智能与机器学习全景	1
2	Python 语言基础	23

1 人工智能与机器学习全景

先修知识、学习目标与本章位置

本章只要求理解函数、平均数和基本代数运算，不要求编程经验。学习后应能区分人工智能的研究范围与不同学习方式；把一个需求改写为输入、目标和评价协议；辨认样本、特征、标签、参数、超参数、损失和指标；解释训练、验证、测试各自承担的工作。

本章给出全书的共同语言。实验先按文字说明和手算理解，代码可在第二章学完函数、列表和循环后回读。近邻方法只作为流程示例，完整算法将在第 15 章讨论。

从一张图片判断物体类别，从一段历史记录预测下一次需求，从一批文档找出相关证据，以及控制一个不断与环境交互的系统，都是人工智能中的典型问题。这些问题的输入、目标和错误代价不同，不能只用“让计算机变聪明”概括。建立机器学习问题的第一步是明确计算机接收什么信息、产生什么输出，以及怎样判断这个输出是否有用。

本章使用两个贯穿例子：根据包裹重量预测配送费用，以及根据一个评分判断消息是否需要进一步检查。所用数值都是为解释概念而构造的教学数据，不代表真实业务规律。第一个例子展示连续预测和数据划分，第二个例子展示阈值与错误类型。

1.1 人工智能、机器学习与深度学习

研究目标与实现方法

人工智能（artificial intelligence, AI）研究能够完成感知、推理、学习、规划、语言处理和决策等任务的计算方法。它包含多种技术路线。搜索算法可以在候选状态之间寻找路径；规则系统可以把已有知识写成条件和结论；机器学习可以利用数据改变模型的行为。一套系统还可以把这些方法组合起来。

设某配送公司已经公布了完整收费规则：起步价为 4 元，每千克增加 2 元。对重量 x ，直接计算 $4 + 2x$ 即可。这里的规则由人给定，无须从数据中估计。若收费还受到拥堵、区域和服务类型影响，无法写出足够准确的固定规则，就可能利用历史记录拟合预测函数。是否采用学习方法，取决于规律是否已知、数据能否获得、误差是否可接受，而不取决于任务名称是否带有“智能”。

定义：机器学习

机器学习（machine learning, ML）研究利用数据或交互经验改进任务表现的方法。一个学习问题至少要交代任务、经验来源和表现标准。改进必须在规定的评价条件下成立，不能只表现为对已见数据记得更准确。

学习过程并不意味着程序自行确定一切。数据来自哪里、目标如何定义、模型允许许多复杂、训练何时停止，通常需要设计者选择。训练算法则在这些约束下，利用数据确定模型中的可调整部分。例如模型形式选为直线以后，程序可以根据历史数据确

定直线的斜率和截距。

深度学习（deep learning, DL）是机器学习中的一类方法，以多层神经网络逐级变换输入表示。以图片为例，输入最初是一组像素数值，网络通过多层计算形成适合任务的表示，最后输出类别得分。层数较多并不自动带来可靠性；数据、训练目标、优化方法和评价协议仍然必要。神经网络也不是所有机器学习模型的统称，决策树、近邻法和线性模型都有各自的表达方式。



该图表示常见研究方法的包含关系，不是系统部件的排他划分。一个语言问答系统可以使用深度网络生成文本，使用传统检索查找文档，再使用人工编写的规则检查输出格式。这些部件并不因为处于同一个产品中就成为同一种算法。

大语言模型的基本位置

语言模型（language model）为文本序列建立概率模型。文本通常先被拆成词元（token），一个词元可能对应一个字、一个词或一个子词片段，不能固定等同于一个汉字。大语言模型（large language model, LLM）通常使用规模较大的深度神经网络，并在大量文本上进行训练。参数规模、训练数据规模和计算量都影响模型，不能仅凭“大”字推出某一具体能力。

以自回归语言模型为例，已知词元序列 $t_1, \dots, t_{j-1}$ ，模型对下一个词元 t_j 给出条件概率

$$p_{\theta}(t_j | t_1, \dots, t_{j-1}).$$

下标 θ 表示模型参数，竖线表示“在前面这些词元给定的条件下”。这里暂不推导概率公式，只需理解：模型先给候选词元分配概率，再按照某种规则选择后续词元，逐步形成文本。输入上下文相同，选择规则不同，生成结果也可能不同。

语言模型的生成目标不直接等同于事实核对。一个句子可能形式完整、语义连贯，却包含错误日期或不存在的文献。因此面向资料问答的系统往往还需要检索、证据组织和引用验证。训练一个模型、调用一个现成模型，以及搭建包含模型的应用系统，是三种不同层次的工作。后续第 29–37 章分别讨论结构、训练、生成和应用评价。

注意：任务、模型与系统

“文本问答”是任务，“Transformer”是一类网络结构，“语言模型”描述建模目标，“检索增强生成”是一种系统组织方式。分析技术方案时，应分别说明这些层次，避免用一个名称替代整个流程。

1.2 学习方式：监督、无监督、自监督与强化学习

学习方式的区别主要来自训练信号如何获得，而不是输入必须是表格、图片还是文字。同一种数据形式可以支持多种学习任务，一套系统也可能先使用一种方式预训练，再使用另一种方式适配具体任务。

监督学习：给定输入与目标的对应关系

监督学习（supervised learning）使用带有目标值的样本。第 i 个样本记为 $(\mathbf{x}_i, y_i)$ ，其中 $\mathbf{x}_i$ 是输入， y_i 是期望预测的目标。例如，包裹重量和区域是输入，最终配送费用是目标；邮件正文是输入，是否为垃圾邮件是目标。

“有标签”只表示存在训练目标，不保证标签准确。标签可以来自人工标注、传感器测量或后续业务记录。人工判断可能不一致，设备可能存在测量误差，业务结果也可能被历史决策影响。若使用“审核是否通过”作为目标，模型学到的首先是历史审核结果的规律，而不是某种不受流程影响的客观属性。

监督学习的关键是输入与标签的对齐。如果一张图片对应了另一张图片的类别，即使每个字段看起来都合法，训练问题仍然错误。后续数据处理中的合并、排序和切片，都必须保留这种对应关系。

无监督学习：没有预先指定的逐样本标签

无监督学习（unsupervised learning）从输入数据中寻找结构，而没有为每个样本提供任务所需的目标标签。把用户按行为相似性分组是聚类，把高维数据压缩到较低维表示是降维。输入之间的距离、重建误差或密度可以形成优化目标。

无监督并不等于没有目标函数，也不等于不需要人作判断。聚类算法可能把一批用户分成三组，但“第三组”本身不是天然存在的业务类别。要解释这些组，仍需检查特征、稳定性和用途。若对不同特征采用不同尺度，得到的相似关系也可能改变。因此一个结构清晰的可视化图并不能独立证明分组有实际意义。

自监督学习：由数据构造训练目标

自监督学习（self-supervised learning）通过数据本身构造训练信号。对一句文本，可以把前半段作为输入，后续词元作为目标；对一张图片，可以遮挡一部分内容并要求模型预测被遮挡部分。目标来自原数据，不需要人为逐条标注新的类别。

自监督训练仍然存在明确的输入、目标和损失。从计算形式看，它常使用与监督学习相似的优化流程；从训练信号的来源看，又与人工标注监督不同。因而不能把监督、无监督和自监督画成在所有定义下都互不相交的三个集合。此处按训练信号来源区分，是为了说明数据如何被使用。

例题 1.1：同一批文本的三种用途

已有一批未标注新闻，另有其中 2000 篇新闻的主题标签。设计三个不同的训练问题，并说明目标从哪里来。

解 第一种是监督分类：输入完整新闻，目标为人工标注的“体育”“科技”等主题，只有带标签的部分直接提供分类监督。

第二种是无监督聚类：输入新闻的某种数值表示，让内容相似的新闻靠近或进入同一组，没有预先要求某组对应哪个主题。聚类结果可以再由人检查和命名。

第三种是自监督语言建模：从原文本截取前面的词元作为输入，将紧接其后的词元作为目标。未标注新闻也能生成这样的训练样本。若随后使用 2000 篇带标签新闻训练分类器，就形成“自监督预训练加监督适配”的两阶段方案。同一篇新闻在不同阶段的角色不同。

强化学习：动作改变后续经验

强化学习 (reinforcement learning, RL) 研究与环境交互的决策。智能体 (agent) 观察当前信息，选择动作 (action)，环境返回新的信息和奖励 (reward)。选择动作的规则称为策略 (policy)。目标通常是提高一段时间内的累计奖励，而不是只预测当前样本的一个标签。

以网格中的路径规划为例，位置是状态，向上下左右移动是动作，到达终点可能获得正奖励，碰到障碍可能受到惩罚。一步绕路可能降低即时奖励，却有助于最终到达目标。这体现了延迟结果和连续决策。为了发现更好的路线，智能体有时必须尝试尚不熟悉的动作，这称为探索；使用目前认为最好的动作则称为利用。

如果只有历史用户记录和一个购买标签，通常首先是监督预测问题。若系统选择展示哪件商品，而这个选择会影响用户后续行为和可观察的数据，就出现交互决策因素。不能仅因系统“做了选择”就断言必须使用强化学习；固定规则、监督模型加阈值，以及搜索规划也可以产生决策。

方式	训练信号	首先需要检查的条件
监督	输入对应的目标值	标签定义、正确性与对齐
无监督	数据结构形成的目标	相似性、尺度与结果解释
自监督	从原数据构造的目标	目标是否泄漏、任务是否有用
强化	交互产生的奖励与转移	奖励含义、延迟与探索条件

1.3 任务类型：回归、分类、生成、检索与决策

回归与分类

回归 (regression) 预测数值型目标，通常允许输出在某个连续范围内变化。预测费用、温度和时长都是回归。目标虽然写成数字，也不一定是回归：邮政编码、班级编号和类别编号的大小一般没有可用于误差计算的连续意义。

分类 (classification) 预测离散类别。两种类别构成二分类，多于两种互斥类别构成多分类。一个样本允许同时属于多个类别时，是多标签问题，例如一张图片同时含

有“树”和“汽车”。多分类通常要求从多个类别中选一个，多标签则需要分别判断各标签是否存在。

分类模型可以先输出得分或概率估计，再通过规则生成类别。设二分类的正类是“需要检查”，得分 $s(\mathbf{x}) \in [0, 1]$ ，选择阈值 τ 后可定义

$$\hat{y} = \begin{cases} 1, & s(\mathbf{x}) \geq \tau, \\ 0, & s(\mathbf{x}) < \tau. \end{cases}$$

符号 $\hat{y}$ 读作“预测的 y ”。阈值是由分数得到行动的规则，并不等于模型内部的所有参数。只有在概率估计经过适当验证时，0.8 才适宜解释为相应条件下约 80% 的发生概率；任意归一化得分不能自动获得这种含义。

生成与检索

生成（generation）产生新的文本、图片、音频或其他结构化对象。输出可能很长，也可能存在多个合理答案。例如对一段文本进行摘要，不能简单要求输出与某一参考摘要逐字相同；评价还需要考察信息是否保留、事实是否一致和内容是否适合任务。

检索（retrieval）从已有集合中找出与查询相关的对象。输入为查询，输出为文档、图片或候选条目的排序。检索输出应当来自待检索集合，而生成输出可以是新组织的内容。检索质量既取决于相关性判断，也取决于候选集合是否包含所需资料。

一个资料问答系统可能先检索若干段落，再根据段落生成答案。此时应分别检查：相关材料有没有被找到；材料是否包含问题答案；生成内容是否忠实于材料；引用是否对应支持该结论的段落。只检查回答是否流畅，无法定位这些不同环节的失败。

预测与决策的联系

决策（decision）是在目标与约束下选择行动。预测可以为决策提供信息，但预测最准确的方案不一定产生最低的业务成本。设需要检查的消息只有 10%，人工检查每条消息都有成本，漏掉危险消息又有另一种成本。怎样选择阈值，取决于这两种错误的相对代价和检查容量。

同一个模型也可能在不同用途下采用不同阈值。如果只是给审核人员排序，重点是靠前条目的相关性；如果直接阻断消息，误判普通消息的代价可能更高。任务说明必须交代模型输出会如何被使用，才能选择适当指标。

例题 1.2：把含糊需求改写成任务

需求为“建立智能配送系统”。给出一个可执行的预测子任务和一个决策子任务，并说明两者的区别。

解 预测子任务可以规定为：在订单被接受时，根据当时已知的重量、区域和服务类型，预测最终配送费用，单位为元；评价新订单上的平均绝对误差。目标明确到一个数值，输入也限定在预测时刻可获取的字段。

决策子任务可以规定为：对当天已确认订单选择配送车辆和路径，在车辆容量、时间窗和可行道路约束下减少总成本。其输出是方案，评价涉及实际成本与约束是否满

足。费用预测可以作为决策中的成本估计，但它不能单独解决路径的组合选择问题。

若不限定预测时点，容易把订单完成后才出现的“实际里程”作为输入，从而得到无法在订单接受时运行的模型。问题定义中的时间条件因此与算法选择同样重要。

1.4 样本、特征、标签与数据表示

逐个样本的记号

样本（sample）是数据中的一个观测单位。订单预测中通常一笔订单是一个样本；图像分类中通常一张图像是一个样本。特征（feature）是用于描述样本、供模型使用的输入信息。标签（label）或目标（target）是监督学习希望预测的量。

对 n 个样本、每个样本 d 个数值特征，常记

$$\mathbf{x}_i = (x_{i1}, \dots, x_{id})^T \in \mathbb{R}^d, \quad i = 1, \dots, n.$$

$\mathbb{R}^d$ 表示 d 维实数向量的集合，上标 T 表示转置。下标 i 选第几个样本， j 选第几个特征， x_{ij} 因而是第 i 个样本的第 j 个输入数值。维度 d 不表示样本数量。

把样本按行排列，得到设计矩阵或特征矩阵

$$X = \begin{bmatrix} x_{11} & \cdots & x_{1d} \\ \vdots & & \vdots \\ x_{n1} & \cdots & x_{nd} \end{bmatrix} \in \mathbb{R}^{n \times d}.$$

本讲义默认行对应样本、列对应特征。回归的目标向量写作 $\mathbf{y} = (y_1, \dots, y_n)^T \in \mathbb{R}^n$ 。具体程序库可能要求目标形状为 $(n,)$ 或 $(n,1)$ ，数学上的同一组数值在程序中仍需检查形状，第 4 章将详细讨论。

订单编号	重量/kg	距离/km	费用/元	字段角色
A01	1.0	3.0	8.0	前两项为特征，费用为标签
A02	2.0	5.0	12.0	编号用于追踪，不默认当特征
A03	1.5	2.0	7.0	每一行是一笔订单

这里 $n = 3, d = 2$ ， X 的形状为 3×2 。编号虽然存放在表格中，却不是自然有预测意义的数值。把 A01 转换成 1、A02 转换成 2，不会自动建立合理的大小关系。类别、文本和图片也可以变成输入表示，但转换规则本身需要设计和验证。

观测单位、重复与缺失

一名用户有十笔订单，意味着十个订单样本，却不意味着十个完全独立的用户。若模型需要预测新用户的行为，同一用户的记录跨入训练和测试可能使测试变得过于容易。若目标是预测既有用户的下一笔订单，按时间评价又可能更合适。数据划分必须与使用情境一致。

缺失值表示某字段未被观测或未被记录，不应未经说明地当成 0。重量为 0 和重量缺失具有不同含义。重复记录也有多种情况：数据导出时重复了一行、同一订单更新了状态、两笔真实订单恰好字段相同。只有理解样本单位，才能决定去重规则。

特征越多也不必然越好。无关字段增加复杂性，有些字段在部署时无法取得，有些字段会把目标的后续结果直接暴露给模型。检查字段的定义、生成时间和可靠性，是模型训练之前的必要工作。

例题 1.3：输入形状与信息可用性

有 1000 笔订单，每笔记录重量、收件区域、下单时刻、最终签收时刻和最终费用。模型在下单时预测费用。哪些字段可以作为候选输入？能否直接说 $d = 4$ ？

解 重量、收件区域和下单时刻在预测时已知，可以作为候选输入；最终签收时刻只有配送完成后才知道，不能用于该预测情境。最终费用是标签。

原始候选字段有三个，但处理后的数值特征维度未必是 3。区域可能需要多个指示变量，下单时刻可能分解为小时与是否休息日。因此必须区分原始字段数和输入向量维度。 $n = 1000$ 是样本数， d 应在具体编码方案确定后再给出。

1.5 模型、参数、超参数与训练

模型是输入到输出的映射

模型（model）用某种规则把输入映射为输出。参数化模型可记为

$$\hat{y} = f_{\theta}(\mathbf{x}).$$

f 表示函数形式， θ 表示可调整的参数（parameter）。对只有重量一个特征的线性预测，

$$\hat{y} = ax + b, \quad \theta = (a, b).$$

x 的单位是千克， $\hat{y}, b$ 的单位是元， a 的单位是元每千克。函数形式限制预测曲线是一条直线，参数决定具体是哪条直线。这种限制是模型假设，不是数据自然保证的事实。

训练（training）是根据训练数据确定模型可调整部分的过程。训练结束后，对新输入计算输出称为推理（inference）或预测。普通推理不需要重新计算全部训练过程；但不同算法保存的信息不同。线性模型可以只保存系数，近邻模型通常需要保存用于查询的训练样本。

模型并不必然通过梯度下降训练。树模型可以选择分裂规则，近邻法可以存储样本并在预测时寻找邻近点。有些学习方法只有少量显式系数，有些方法保存复杂结构。因而“训练就是不断修改神经网络权重”只适用于其中一类方法。

损失、经验风险与优化目标

损失函数（loss function）量化一次预测的错误，例如平方损失

$$\ell(\hat{y}, y) = (\hat{y} - y)^2.$$

误差 $\hat{y} - y$ 的正负分别表示高估和低估，平方后两种方向都产生非负代价，大误差增长更快。若费用用元计，平方损失的单位为元的平方，不能直接把它解释为平均多收了多少元。

把损失在训练样本上平均，得到经验风险（empirical risk）

$$\hat{R}_{\text{train}}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(\mathbf{x}_i), y_i). \quad (1.1)$$

求和符号表示逐个样本累加； n 为参与该次平均的样本数；每项损失为一个标量，因此结果也是标量。训练算法通常试图降低这个目标，或降低加入复杂度惩罚后的目标。

优化（optimization）研究怎样寻找使目标函数较小的参数。学习除了优化，还包括模型与数据能否支持对新样本的预测。训练损失很低但新样本表现差，可能是泛化问题；训练损失始终很高，则也可能是模型限制、数据错误或优化失败。两种现象需要不同的诊断。

例题 1.4：平方误差的完整计算

三笔订单的重量为 (1, 2, 3)，真实费用为 (6, 8, 9)。模型为 $\hat{y} = 2x + 4$ 。计算预测、逐样本误差、平均平方损失和平均绝对误差。

解 依次代入输入，得到预测 (6, 8, 10)。预测减真实值为 (0, 0, 1)，说明前两项恰好吻合，第三项高估 1 元。平方损失为 (0, 0, 1)，因而

$$\text{MSE} = \frac{0^2 + 0^2 + 1^2}{3} = \frac{1}{3} \text{ 元}^2, \quad \text{MAE} = \frac{|0| + |0| + |1|}{3} = \frac{1}{3} \text{ 元}.$$

数值恰好相同是该数据的偶然结果，二者单位和对大误差的敏感程度不同。如果第三项误差改成 3 元，MSE 变成 3 元²，MAE 变成 1 元。

超参数与选择规则

超参数（hyperparameter）是训练流程或模型结构中需要预先设定、通常由验证方案选择的设置，例如近邻法的邻居个数 k 、树允许的最大深度、梯度下降的学习率。参数由某次训练决定，超参数则在不同训练方案之间进行选择。这是相对于具体训练流程的区分，不能仅通过某个数的名字判断。

例如，直线的 a, b 在通常线性回归中是参数；若人为规定几种固定斜率、只让程序估计截距，则斜率已成为候选方案的一部分。应按它在当前流程中是否被拟合来解释。另一个常见误区是把“训练样本数”与“迭代次数”混淆：前者来自数据，后者描述训练算法重复计算的次数。

概念	线性预测中的例子	作用
模型形式	$ax + b$	限制输入到输出的函数族
参数	训练得到的 a, b	决定具体预测函数
超参数	正则化强度等设置	决定训练或结构方案
训练算法	求解最小二乘等方法	寻找符合目标的参数
指标	测试 MAE	概括指定数据上的表现

“把指标作为训练目标”在某些任务中可行，但二者角色仍不同。训练目标强调可优化性，评价指标强调任务用途。分类中的某些平滑损失有利于训练，最终却可能以准确率、召回率或其他指标比较模型。

1.6 怎样从实际需求定义学习问题

任务说明的六个要素

一个可执行的任务说明应包含样本单位、预测时点、输入范围、目标定义、评价总体和错误代价。下面逐项解释。

样本单位决定一行数据代表什么。一行是“订单”还是“订单中的一个包裹”，会改变数据量、标签和分组关系。预测时点决定可用信息，例如下单时预测费用与配送完成后检查账单，是两个不同任务。

输入范围说明哪些字段可读，哪些字段必须屏蔽。目标定义要给出测量方法、单位和时间范围。预测“未来销量”必须说明是下一小时、下一天还是下一月，是否包含取消订单。评价总体说明需要对哪些新数据有效：相同城市的后续订单、新城市订单，或全新用户。错误代价则把统计误差与业务使用联系起来。

任务卡：下单时费用预测

观测单位是一笔已接受订单。预测在下单确认后、开始配送前进行。候选输入为当时已知的重量、区域、服务类型和下单时间；目标为订单最终结算费用，单位为元。评价对象为同一服务范围内后续新增订单。主要报告 MAE，同时检查高费用订单的误差；先与只预测训练平均费用的基线比较。

该任务卡还没有说明算法，但已经排除了许多不合理做法。如果未来应用是进入新城市，必须增加城市之间的评价；如果某种服务只出现在测试期，需讨论模型是否有处理它的表示。问题定义往往比模型名称更早决定实验是否可信。

基线、数据来源与标签构造

基线 (baseline) 是作为比较起点的简单方案，例如回归中始终预测训练平均值，分类中始终预测多数类，检索中使用简单关键词匹配。基线可以暴露数据难度和指标陷阱。复杂模型如果没有优于合适基线，首先应检查它是否带来实际价值。

只预测训练均值的方案记为 $\hat{y} = c$ ，其中

$$c = \frac{1}{n} \sum_{i=1}^n y_i.$$

这个常数由训练数据确定，不能使用测试标签计算。平均值是平方损失下的最优常数：记训练均值为 $\bar{y}$ ，对任意常数 c ，展开平方可得

$$\begin{aligned} \sum_i (y_i - c)^2 &= \sum_i [(y_i - \bar{y}) + (\bar{y} - c)]^2 \\ &= \sum_i (y_i - \bar{y})^2 + 2(\bar{y} - c) \sum_i (y_i - \bar{y}) + n(\bar{y} - c)^2 \\ &= \sum_i (y_i - \bar{y})^2 + n(\bar{y} - c)^2. \end{aligned}$$

中间和为零，因为 $\sum_i y_i = n\bar{y}$ ；最后一项非负，取 $c = \bar{y}$ 时为零。如果目标改成绝对损失，最优常数一般为训练标签的中位数，不能沿用上述平方展开作证明。本章实验将均值作为固定基线，同时用 MAE 评价；这是一种明确的比较约定，不宣称它在 MAE 下最优。

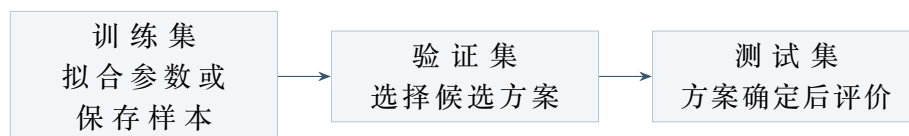
数据来源需要记录采集条件。人工筛选出的“正常订单”可能遗漏异常场景；只记录已经完成的订单可能排除取消订单；历史审核数据可能偏向曾经被检查的样本。清洗不能简单等同于删除难例，因为这些难例可能恰好是模型实际会遇到的情况。

标签还要说明延迟。订单是否最终送达、用户是否在 30 天内购买，只有等待相应时间之后才能确定。把尚未观察完整的记录一律标成“未发生”，会混入错误标签。应按数据截止时间和观察窗口处理。

1.7 训练、验证、测试与泛化

三类数据的职责

训练集 (training set) 用于拟合模型可调整部分。验证集 (validation set) 用于选择候选模型、超参数和决策阈值。测试集 (test set) 用于在方案确定后，给出对新数据表现的最终评估。三者的区别是用途，通常不能只靠文件名称保证。



例如比较近邻个数 1、3 和 5，三个候选都只使用训练样本预测，在验证集上比较 MAE。选定 k 后再计算测试 MAE。如果观察测试结果后继续更换 k ，测试集已经参与选择，它的分数不再是一次未参与选择的独立最终评价。

最终方案有时会在训练集与验证集合并后重新拟合，再在保留的测试集上评价。这需要在协议中预先规定，尤其要说明合并后重新拟合的具体内容。本章实验为避免混淆，选定 k 后保持原训练集，不做合并重训。

泛化、欠拟合与过拟合

泛化 (generalization) 指模型对未参与相应拟合或选择的新样本的表现。训练集只是一批有限观测，不能列举未来所有输入。理想目标是降低目标使用环境中的平均错误，而不只是降低训练数据上的平均错误。

欠拟合 (underfitting) 表示模型或训练过程未能捕捉数据中与任务有关的规律，常表现为训练与验证都不好。过拟合 (overfitting) 表示模型对训练数据中的偶然细节适应过强，新数据表现较差。训练好、验证差是需要检查的信号，但也可能来自错误划分、分布变化或实现不一致，不能只根据两行分数断言原因。

一个极端模型可以记住每个训练编号及其费用。如果预测任务中的新订单有新编号，记忆便无法直接提供有用预测。训练误差为零只说明存储或拟合了已见数据。具有较强表达能力的模型需要足够数据、合理限制和可靠评价，才能判断其优势。

从统计观点，常假设训练和目标新样本来自同一分布，并且样本之间具有适当独立性。该假设有助于把有限样本表现与总体表现联系起来，但现实中可能不满足。费率调整、城市改变、用户行为变化，都可能引起分布变化 (distribution shift)。遇到这种变化，即使先前评价完全合规，模型也可能失效。

划分方式应模拟实际使用

随机划分适合样本近似独立、未来使用条件与当前样本混合分布一致的情况。时间划分用较早记录训练、较晚记录评价，更接近预测未来。分组划分让同一用户、设备或病人的记录只进入一个集合，用于评价对新组的泛化。分类中分层划分可尽量保留各集合的类别比例，但它不能替代时间或分组约束。

教学中常见的比例不是普遍规则。数据少时，固定留出集分数可能波动很大；测试集中少数类过少时，召回率可能缺乏稳定性。第 14 章将使用交叉验证和不确定性分析处理这些问题。本章重点在于先决定各集合的用途，而不是背诵一个固定百分比。

注意：数据泄漏

数据泄漏 (data leakage) 指拟合或选择过程使用了在规定预测情境中不应获得的信息。包括目标产生后的字段、跨集合的重复或关联记录，以及用全部数据估计预处理规则。泄漏使评价条件偏离实际使用条件，分数可能因此过于乐观。

即使标准化没有读取标签，也不应在常规独立测试协议下用测试数据估计均值和方差。它仍然让训练过程利用了测试输入的分布信息。通常先划分，再用训练集拟合预处理规则，最后将同一规则应用到验证和测试。具体算法和特殊协议可能不同，必须明确说明。

例题 1.5：一次高分评价的排查

某模型训练准确率 99%，测试准确率 98%。训练与测试按照片随机划分，其中同一个人的连续拍摄照片出现在两边。应用目标是识别从未出现过的新人的某种表情。这个分数能否直接证明对新人有效？

解 不能。连续照片共享人物外观、背景和拍摄条件，样本之间有明显关联。照片级随机划分可能让模型在测试时遇到与训练非常接近的输入，而应用要求对新人物有效。

应按人物分组，将某个人的照片全部放在同一集合，重新训练与评价。若需要在新场景使用，还应考虑拍摄场景是否也跨集合。99% 与 98% 的接近只能说明当前划分下两组表现接近，无法消除划分与实际目标不一致的问题。

1.8 评价指标与错误的含义

回归指标：MAE、MSE 与 RMSE

对评价集中的 m 个样本，真实值为 y_i ，预测为 $\hat{y}_i$ ，常用

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |\hat{y}_i - y_i|, \quad (1.2)$$

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2, \quad (1.3)$$

$$\text{RMSE} = \sqrt{\text{MSE}}. \quad (1.4)$$

MAE 是平均绝对误差，MSE 是均方误差，RMSE 是均方根误差。这里 m 专指当前评价集合的大小，避免与训练样本数混淆。MAE 和 RMSE 具有与目标相同的单位，MSE 具有目标单位的平方。三者都越小越好，但比较时必须使用相同样本、目标单位和计算约定。

如果误差为 (0,0,0,4)，MAE 为 1，RMSE 为 2。若误差为 (1,1,1,1)，两者都为 1。MAE 在这两个方案间打平，RMSE 则更偏好均匀的小误差，因为平方对单个大误差施加更强代价。选择指标因此隐含了对错误分布的偏好。

只看平均值还可能掩盖子群体差异。例如整体 MAE 较小，但远距离订单误差很大，部署到以远距离订单为主的区域仍会失败。应结合分组结果和具体错例；分组指标样本少时，也要同时报告样本数，不能将偶然变化解释为稳定规律。

分类指标：先数清四种结果

二分类中先规定正类，例如“需要检查”。真阳性（true positive, TP）是实际正类且预测正类；假阳性（false positive, FP）是实际负类但预测正类；真阴性（true negative, TN）是实际负类且预测负类；假阴性（false negative, FN）是实际正类但预测负类。

	预测正类	预测负类
实际正类	TP	FN
实际负类	FP	TN

这个表称为混淆矩阵（confusion matrix）。不同资料可能把实际与预测放在相反轴上，因此读取矩阵时必须先看轴标，不能只凭位置认定哪一格是 FP。

准确率 (accuracy)、精确率 (precision) 和召回率 (recall) 定义为

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}, \quad (1.5)$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (1.6)$$

$$\text{Recall} = \frac{TP}{TP + FN}. \quad (1.7)$$

准确率回答“所有判断中有多少正确”；精确率回答“判为正的样本中有多少确实为正”；召回率回答“实际为正的样本中找出了多少”。精确率与召回率的分母不同，不能互换。

若模型从不预测正类，精确率分母为零，数学上未定义；若评价集中没有实际正类，召回率分母为零。程序必须明确处理约定，例如返回缺失标记并报告原因。不能悄悄把未定义值当成真实的 0 或 1，然后与正常指标混合比较。

例题 1.6: 多数类与准确率陷阱

100 条消息中，实际有 5 条需要检查。方案 A 把全部消息判为不需要检查；方案 B 找出其中 4 条，但同时错报 10 条普通消息。比较准确率、精确率与召回率。

解 方案 A 的 TP 为 0、FP 为 0、TN 为 95、FN 为 5，因此准确率为 95%，精确率未定义，召回率为 0。高准确率来自负类占多数，不能证明模型找到了需要检查的消息。

方案 B 的 TP 为 4、FP 为 10、TN 为 85、FN 为 1，准确率为 $(4 + 85)/100 = 89\%$ ，精确率为 $4/14 \approx 28.57\%$ ，召回率为 $4/5 = 80\%$ 。方案 B 总体准确率较低，却覆盖了大部分正类，同时带来更多人工检查负担。是否有用，需结合漏检与误报成本判断。

阈值不是模型质量的唯一刻度

提高正类阈值通常减少被判为正的样本，因此 TP 与 FP 都不会增加，召回率通常下降。精确率却不保证单调提高：如果被删除的样本中真阳性的比例高于保留部分，精确率也可能下降。它的实际变化取决于得分排序和标签。

若要求每天最多检查 100 条消息，就还需要考虑候选数量和排序。若要求漏检率不高于某个水平，应根据验证数据选择阈值，并在未参与选择的数据上核对。阈值的选择属于方案选择，不能在测试集上反复尝试后仍把最好结果当成独立最终评价。

本章只讨论三个基本分类指标。F1、ROC、PR 曲线、校准和区间估计会在第 14 章展开。现阶段的重点是能把指标的每一个分母与具体样本集合对应起来。

1.9 完整小案例：从数据到最终评价

任务、教学数据与比较方案

假设需要在包裹进入配送系统时，仅根据重量预测费用。简化输入 x 的单位为 kg，输出 y 的单位为元。教学数据在约 0.5–6.0 kg 范围内构造，费用大致随重量增加，同时含有小幅偏差。真实费用还受到许多因素影响，本例刻意只保留一个特征，以便手算和逐步追踪。

本例预先固定训练 12 条、验证 4 条、测试 4 条。它不是从真实总体随机抽样得到，

因此只能展示协议和计算，不能据此估计真实系统的预期误差。后续项目中必须根据实际采集方式、时间和分组关系建立划分。

训练集：前半		训练集：后半		验证集	
x/kg	$y/\text{元}$	x/kg	$y/\text{元}$	x/kg	$y/\text{元}$
0.5	5.2	3.5	11.4	0.8	5.7
1.0	5.6	4.0	11.7	2.2	8.2
1.5	7.1	4.5	13.1	3.8	11.8
2.0	8.3	5.0	14.4	5.2	14.3
2.5	8.6	5.5	14.7		
3.0	10.2	6.0	16.1		

测试输入为 1.2, 2.8, 4.2, 5.8 kg，相应标签为 6.5, 9.5, 12.3, 15.5 元。标签虽在讲义中公开，实际执行方案仍须坚持先完成验证选择，再计算测试结果。公开教学数据适于核对代码，不适于模拟长期保密测试集。

比较方案包括均值基线和一个简单的近邻回归规则。对新重量 x ，计算它与每个训练重量的距离 $|x - x_i|$ ，选最近的 k 个训练样本，取这些样本费用的平均值：

$$\hat{y}(x) = \frac{1}{k} \sum_{i \in N_k(x)} y_i.$$

$N_k(x)$ 是选中训练样本的下标集合，包含 k 项。距离与重量单位相同，费用平均的单位仍为元。候选 k 预先规定为 1、3 和 5；验证 MAE 越低越好，同分时取较小的 k 。距离同分时保留训练表中的先后次序，代码中的稳定排序实现这一规则。

手算一次预测与基线

验证输入 $x = 2.2$ 到训练重量 2.0、2.5 和 1.5 的距离分别为 0.2、0.3 和 0.7，其他距离更大。因此

$$\hat{y}_{k=1} = 8.3, \quad \hat{y}_{k=3} = \frac{8.3 + 8.6 + 7.1}{3} = 8.0.$$

真实值为 8.2，所以两种预测的绝对误差分别是 0.1 和 0.2 元。该点上 $k = 1$ 更好，但选 k 不能只观察这一点，要对整个验证集平均。

训练费用总和为 126.4 元，训练均值为

$$c = \frac{126.4}{12} \approx 10.533333 \text{ 元}.$$

均值基线对每个输入都输出这个数。它忽略重量，却利用训练集的总体费用水平。验证集上的四项绝对误差约为 4.833333、2.333333、1.266667、3.766667 元，平均为 3.05 元。

候选选择与最终报告

将每个候选对四个验证样本的误差平均，得到

方案	验证 MAE/元
均值基线	3.050000
近邻 $k = 1$	0.100000
近邻 $k = 3$	0.241667
近邻 $k = 5$	0.540000

依照预定规则选 $k = 1$ 。保持原训练集，在测试集上预测，得到

x/kg	真实费用/元	预测费用/元	预测减真实/元	绝对误差/元
1.2	6.5	5.6	-0.9	0.9
2.8	9.5	10.2	0.7	0.7
4.2	12.3	11.7	-0.6	0.6
5.8	15.5	16.1	0.6	0.6

最终测试 MAE 为 $(0.9 + 0.7 + 0.6 + 0.6)/4 = 0.7$ 元，均值基线测试 MAE 为 2.95 元。验证与测试 MAE 不同是正常现象，它们包含不同样本。测试只有 4 条，更不能将 0.7 元当作某个真实总体的精确平均误差。

这个结果支持的结论是：在当前固定教学划分上，按验证集选出的近邻规则优于指定均值基线。它不支持“近邻必然优于线性回归”，因为没有进行相应比较；也不支持“实际配送误差不超过 0.7 元”，因为平均误差不限制每个样本，更没有实际数据证据。

适用边界与可复现步骤

当输入 $x = 10 \text{ kg}$ 时， $k = 1$ 仍取最接近的 6.0 kg 训练样本，输出 16.1 元。它不会根据重量继续增长自动推算费用。输入落在已见范围之外的预测称为外推 (extrapolation)；这种边界行为提醒模型有自己的归纳规则，不能只因范围内结果好就信任范围外结果。

本例的执行顺序为：固定任务与数据划分，拟合基线或保存训练记录，计算验证指标，选定候选，计算一次最终测试指标，检查逐样本误差并说明局限。其完整预测计算通过对 n 个训练距离排序完成，每个查询通常需要 $O(n \log n)$ 排序时间，保存训练表需要 $O(n)$ 空间。这里的记号只表示增长趋势：数据规模翻倍，排序成本会增长；不是精确运行秒数。第 15 章将讨论更高效的实现。

注意：简化实现的边界

本例只有一个数值特征、20 条构造记录，无缺失值和复杂编码。它展示完整学习流程的骨架。实际项目仍需要数据质量检查、合适划分、预处理、更多基线、统计不确定性和部署后的监测。

1.10 实验 1.1：可运行的近邻预测流程**环境、输入与运行**

代码入口为 `ch01_regression.py`，保存在“配套实验”目录。使用 Python 3 标准库，不读取外部文件，不下载数据，不安装第三方库。在该目录打开终端后运行：

```
python3 ch01_regression.py
```

Windows 环境若 Python 由启动器管理，可用 `py -3 ch01_regression.py`。解释器和终端的区别在第 2 章说明。此时可以先看输出、手算核对，再于第二章后阅读函数细节。

数据列表与上一节完全对应。核心函数如下：

```
1 def neighbor_predict(data, x, k):
2     if not 1 <= k <= len(data):
3         raise ValueError("invalid k")
4     ordered = sorted(data, key=lambda row: abs(row[0] - x))
5     selected = ordered[:k]
6     return sum(y for _, y in selected) / k
7
8
9 def mae(data, predict):
10     errors = [abs(predict(x) - y) for x, y in data]
11     return sum(errors) / len(errors)
```

`data` 包含若干 (x_i, y_i) 对。第 2 行检查 k 是否在 1 到训练数量之间；第 4 行按距离排序，`row[0]` 取训练重量；第 5 行只保留前 k 条；第 6 行把费用相加后除以 k 。`lambda` 是一个短的匿名函数，这里只表达“按重量距离排序”，无需在本章掌握语法。

`mae` 接受一个预测函数，对评价样本逐个计算绝对误差，最后平均。这样指标函数不必知道预测来自均值、近邻还是其他模型。代码中的函数式组织会在第二章函数一节再次分析。

选择逻辑与预期输出

```

1 results = []
2 for k in [1, 3, 5]:
3     score = mae(valid, lambda x: neighbor_predict(train, x, k))
4     results.append((score, k))
5 _, best_k = min(results)
6 final_score = mae(
7     test, lambda x: neighbor_predict(train, x, best_k)
8 )

```

每个候选使用同一份 `train`。验证标签只进入误差计算，未被传给近邻预测函数。`results` 保存“验证分数、邻居数”对；`min` 按元组先比较分数，再比较 k ，实现预定同分规则。最终测试行位于选择之后。

完整脚本预期主要输出为：

```

split sizes: 12 4 4
baseline validation MAE: 3.050000
k=1 validation MAE: 0.100000
k=3 validation MAE: 0.241667
k=5 validation MAE: 0.540000
selected k: 1
baseline test MAE: 2.950000
selected model test MAE: 0.700000

```

随后还会打印四个测试样本的预测和误差。输出保留六位小数是显示约定，不表示原始费用测量有六位小数精度。使用相同数据和规定排序规则时，应得到相同的打印结果；浮点内部结果与手算小数允许约 10^{-9} 的绝对差异。

修改任务与结果解释

先将训练标签的第二项 5.6 改成 9.6，重新运行，并手算验证输入 0.8 和测试输入 1.2 的最近邻预测。修改的是训练数据，允许重新执行完整选择；应记录这一变化，不能把新输出与原实验视为同一次固定评价。

再增加一个待预测重量 10.0，只打印预测，不把它混入原测试 MAE。该输入没有真实标签，不能计算正确误差。观察输出是否随重量继续线性增长，并联系上一节的外推限制。

最后尝试令 $k = 0$ 或 $k = 13$ ，确认程序主动拒绝不合法邻居数。如果删去检查， $k = 0$ 会导致除以零； $k = 13$ 则只有 12 个样本被选中，却错误地除以 13。这说明边界检查不仅为防止程序崩溃，也为防止产生看似正常但数学意义错误的数值。

1.11 实验 1.2：阈值、混淆矩阵与检查成本

输入与目标

代码入口为 `ch01_classification.py`。八条消息的标签与固定分数为

下标	0	1	2	3	4	5	6	7
标签	1	1	0	0	1	0	0	0
分数	.90	.60	.80	.30	.40	.20	.55	.10

标签 1 表示需要检查，0 表示不需要。分数在本实验中是人为给定的示意值，不宣称具有已校准概率含义。实验不训练分类模型，只分析从固定分数到类别的过程。因此本表也不是用于选定真实业务阈值的最终测试集。

先以阈值 0.5 手算预测：下标 0、1、2、6 判为正，其余判为负。逐条对照标签得到 TP=2、FP=2、TN=3、FN=1。阈值 0.7 时只有下标 0、2 判为正，得到 TP=1、FP=1、TN=4、FN=2。

实现与逐条核对

```

1  tp = fp = tn = fn = 0
2  for label, score in zip(labels, scores):
3      predicted = int(score >= threshold)
4      if label == 1 and predicted == 1:
5          tp += 1
6      elif label == 0 and predicted == 1:
7          fp += 1
8      elif label == 0 and predicted == 0:
9          tn += 1
10     else:
11         fn += 1

```

zip 将相同位置的标签和分数配成一对。score >= threshold 得到真假值，转成整数后真为 1、假为 0。四个分支互斥，每条有效二分类记录恰好使一个计数加 1。完整脚本在循环前检查长度一致且非空，本实验输入预先保证标签属于 0 或 1。

运行

```
python3 ch01_classification.py
```

得到

```

threshold: 0.5
TP FP TN FN: 2 2 3 1
accuracy=0.625000 precision=0.500000 recall=0.666667
threshold: 0.7
TP FP TN FN: 1 1 4 2
accuracy=0.625000 precision=0.500000 recall=0.333333

```

两个阈值准确率都为 5/8，精确率都为 1/2，召回率却从 2/3 降至 1/3。因此只观察准确率，可能看不出漏检增加。所有计数应与手算完全一致；分数仅存在打印舍入差异。

修改任务

假设一次误报成本为 1，一次漏检成本为 5，定义总错误成本为 $C = FP + 5FN$ 。阈值 0.5 的成本为 7，阈值 0.7 的成本为 11。成本单位是人为定义的统一代价，不能在没有任何实际业务估计时当成真实金额。

将下标 4 的分数由 0.4 改成 0.6，其标签仍为 1。先不运行程序，分别预测两个阈值下四个计数怎样变化，再运行核对。还可将阈值设为 1.1，检查“没有预测正类”的情形：精确率返回 `None`，召回率为 0。完整脚本的固定格式打印为两个正常阈值设计；扩展到未定义精确率时，应先判断 `None` 再决定显示字符串，不能直接套用小数格式。

1.12 知识联系、常见误区与自检

从问题到结果可用一条因果关系清楚的工作链描述：任务决定可用信息与目标；目标和错误成本决定评价；数据采集与划分决定评价条件；模型和训练算法决定候选预测规则；验证决定选择；测试提供在该协议下的结果。这条链中任何一环出现偏差，都可能使精确计算得到的分数失去原本含义。

本章常见误区有以下几类。把 AI 与神经网络等同，会忽略搜索和规则等方法；把所有数字目标当成回归，会给类别编号施加虚假的大小关系；把训练损失低当成新数据可靠，会忽略泛化；把验证和测试互换，会让最终结果受到选择影响；把评分当成概率，会误解阈值；把平均误差当成最大误差，会低估异常样本风险。

模型表现还不是因果结论。若较重的包裹往往同时使用更远的路线，模型可能利用重量预测费用，但这不证明“仅增加重量就会使费用按同样规律变化”。预测描述已观测关系，因果问题需要额外假设、设计或证据。本书主线首先建立预测建模与可靠评价基础。

完成本章后，应能不看公式解释 MAE 分母是什么；用具体记录说明 FP 与 FN；给出一个预测时不可用的字段；解释为何选 k 要读验证集而非测试集；说明一个实验结论的适用范围。如果只能复述名词而无法对新场景作上述判断，应回到任务卡和两个实验逐项对应。

1.13 分层习题

以下题目编号为 1.1–1.12。概念题要求用完整任务说明回答，计算题需写出中间量，实验题需同时给出数值与结论的边界。

- 1.1 概念。固定规则判断银行卡号格式、从标注图片学习识别缺陷、对文本预测下一个词元、在网格中根据奖励学习移动，分别对应什么方法或学习方式？能否把固定规则排除出所有人工智能系统？
- 1.2 任务定义。“预测学生成绩”过于含糊。以学期开始时预测期末分数为目标，写出样本单位、预测时点、三个候选输入、一个不能使用的字段和一个评价指标。说明标签单位。
- 1.3 数据表示。200 张图像已被表示为每张 64 个数值，标签有 5 类。写出 X 的形状；

- 类别标签必须表示为 200×5 矩阵吗？若采用整数类别编号，怎样解释编号？
- 1.4 计算。真实值为 (2, 4, 6)，预测为 (3, 4, 4)。计算残差、MAE、MSE 与 RMSE，并说明目标单位为元时各指标的单位。
- 1.5 训练目标。标签为 (2, 2, 8)。求平方损失下最佳常数预测与相应 MSE；再比较常数 2 与常数 4 的 MAE。解释为什么均值基线不一定是 MAE 最优基线。
- 1.6 模型选择。候选 A、B、C 的训练 MAE 分别为 0.2、0.5、0.8，验证 MAE 分别为 1.5、0.7、0.9。按最小验证 MAE 规则选哪个？能否从这些分数确定 A 一定发生了过拟合？
- 1.7 泄漏判断。预测订单最终费用时，分别讨论使用最终付款状态、在全部数据上拟合填补均值、同一订单复制记录跨入训练和测试。写出每种情况的修正措施。
- 1.8 分类计算。20 个实际正类、80 个实际负类中，模型找到 15 个正类，同时错报 12 个负类。给出混淆矩阵、准确率、精确率与召回率。将正类改成“普通消息”后，原 FP 对应新的哪种错误？
- 1.9 划分。同一用户有多条记录，数据跨越两年。任务甲是预测已有用户的下一次购买；任务乙是预测全新用户是否购买。分别提出合适的主要划分约束，说明为什么随机按行划分不一定合理。
- 1.10 手算近邻。训练记录为 (1, 5), (2, 7), (4, 11), (5, 13)。对 $x = 3.2$ 计算 $k = 1$ 和 $k = 3$ 的预测；若真实值为 10，比较绝对误差。对 $x = 8$ 的 $k = 1$ 预测又是多少？
- 1.11 实验。在实验 1.2 中把下标 4 的分数改成 0.6，其他不变。分别计算阈值 0.5 和 0.7 的指标、成本 $FP + 5FN$ ，并运行程序核对。若要选择真实系统阈值，还需要哪些数据安排？
- 1.12 综合。某报告写道：“尝试 100 次后选出测试准确率最高的模型，准确率 97%，因此任何未来消息都至少有 97% 概率被正确处理。”指出报告中的两个不同错误，并改写为条件明确的报告流程。

1.14 习题参考答案

1.1–1.3：概念与数据

1.1 银行卡号格式判断使用人工给定的规则，不必从样本学习；缺陷识别是监督分类；下一词元预测是从文本构造目标的自监督学习；按交互奖励学习移动是强化学习。固定规则可以成为 AI 系统的一部分，例如约束输出、组织推理或校验数据，但不能仅因使用计算机和条件语句，就把任意普通程序都赋予一个有实质意义的 AI 能力结论。

1.2 样本单位为某个学期的一名学生，预测时点为学期开始前或开始时，需明确到数据可用截止时刻。候选输入可以是先修课程成绩、已完成课程数量、当前课程类别；期末考试得分本身、期末后的教师评分与全学期出勤汇总均不能作为此时输入。标签为该学期最终分数，假设满分 100，单位为分。可用 MAE 评价，另检查不同课程或历史水平群体的误差。该示例是任务定义练习，不假设这些输入足以产生可靠预测。

1.3 $X \in \mathbb{R}^{200 \times 64}$ 。标签可以是长度 200 的整数序列，每项属于 $\{0, 1, 2, 3, 4\}$ ；也可以根据算法需要使用其他编码，并非必须为 200×5 。整数是类别代号，不意味着类别 4 是类别 2 的两倍，也不宜对代号直接计算回归误差。

1.4–1.6: 损失与选择

1.4 以预测减真实定义残差，得到 $(1, 0, -2)$ 。绝对值为 $(1, 0, 2)$ ，平方为 $(1, 0, 4)$ ，所以

$$\text{MAE} = 1, \quad \text{MSE} = \frac{5}{3}, \quad \text{RMSE} = \sqrt{\frac{5}{3}} \approx 1.2910.$$

MAE 与 RMSE 单位为元，MSE 单位为元²。残差平均为 $-1/3$ ，它不能代替 MAE，因为正负错误可能抵消。

1.5 训练均值为 $(2 + 2 + 8)/3 = 4$ 。平方误差为 $(4, 4, 16)$ ，MSE 为 8。常数 2 的 MAE 为 $(0 + 0 + 6)/3 = 2$ ，常数 4 的 MAE 为 $(2 + 2 + 4)/3 = 8/3$ 。均值优化平方损失，中位数 2 在本例中给出更小的绝对损失。两种目标对偏离较大的 8 赋予不同权重。

1.6 选 B，因为验证 MAE 为 0.7，三者中最低。A 训练很低但验证高，符合需要排查过拟合的现象，也可能受划分偏差、分布变化或预测实现错误影响。应检查流程后再诊断，不能仅靠两个指标证明唯一原因。C 训练与验证都较高，可能模型能力不足或训练不充分，也要结合具体目标判断。

1.7–1.9: 评价条件

1.7 最终付款状态产生在预测时点之后，应屏蔽该字段或重新定义预测时点。全数据填补均值把测试输入信息用于处理规则，应先划分，仅从训练集估计均值，再应用到其他集合。复制的订单跨集合会使测试包含训练已见实例，应先识别重复并按订单标识整体分配。去重规则需保留真正不同的订单，不能只因重量相同便删除。

1.8 $TP=15$ 、 $FN=5$ 、 $FP=12$ 、 $TN=68$ 。准确率为 $83/100 = 83\%$ ，精确率为 $15/27 \approx 55.56\%$ ，召回率为 $15/20 = 75\%$ 。若正负类整体交换，原来“实际负、预测正”的 FP 变成“实际正、预测负”的 FN。这说明错误名称依赖正类约定。

1.9 任务甲应以时间为主要约束，训练用较早记录，评价用后续记录；同一用户可按任务需要在前后时期出现，但只能使用截止时刻以前的信息。任务乙应以用户分组为主要约束，测试用户不能在训练中出现；若部署还面向后续时间，应同时设计时间边界。随机按行会混合前后时期，也会让同一用户跨集合，可能偏离两个任务中要求的条件。

1.10–1.12: 计算、实验与报告

1.10 对 3.2，四个距离依次为 2.2、1.2、0.8、1.8。最近一个是 $(4, 11)$ ，所以 $k = 1$ 预测 11，误差 1。最近三个依次为 $(4, 11)$ 、 $(2, 7)$ 、 $(5, 13)$ ，预测 $31/3 \approx 10.3333$ ，误差 $1/3$ 。该点上 $k = 3$ 更好，不能从一个点决定全局最优。对 8，最近为 $(5, 13)$ ， $k = 1$ 预测 13，表现为边界常值而非继续增长。

1.11 阈值 0.5 时，下标 4 由漏检变为检出， $TP=3$ 、 $FP=2$ 、 $TN=3$ 、 $FN=0$ 。准确率

$6/8 = 0.75$ ，精确率 $3/5 = 0.6$ ，召回率 1，成本 2。阈值 0.7 时该样本仍被判负，因此计数与原实验相同，准确率 0.625，精确率 0.5，召回率 $1/3$ ，成本 11。真实系统需要独立训练与验证数据，在验证集选阈值，保留测试集作最终评价，并确保类别比例、采集条件和成本定义与实际使用相符。

1.12 第一个错误是反复使用测试集选模型，它已经成为选择数据；第二个错误是把有限集合上的平均正确比例解释为每个未来消息都满足的概率下界。未来分布可能变化，个体错误概率也不相同。

可先固定任务、划分和指标，用训练集拟合，用验证集完成 100 次候选比较，记录选择规则；选定方案后只作一次保留测试评价，并报告测试样本数、类别比例、指标和采集条件。若测试数据已被用于比较，需要另行准备未参与选择的评价数据。报告只陈述指定协议下观察到的结果，再分析不确定性与适用边界。

1.15 本章小结

机器学习通过数据或交互经验改进任务表现，深度学习和大语言模型是其中的重要方法与模型类别。学习方式取决于训练信号来源，任务类型取决于希望获得的输出。样本、特征、标签定义了数据，模型、参数、训练目标和算法定义了拟合过程，验证与测试定义了选择和最终评价的职责。

实际任务必须交代预测时点、可用字段、目标单位、评价对象和错误代价。训练表现、有限测试表现与未来总体表现是不同对象。下一章建立运行和阅读实验所需的 Python 基础，将本章的逐样本计算变成可核对的程序。

2 Python 语言基础

先修知识、学习目标与本章位置

本章从运行程序开始，只要求基本算术和第一章中的样本、误差概念。学习后应能区分终端、解释器与编辑器；运行并修改脚本；使用基本类型、条件、循环和容器；编写具有明确输入、输出和边界条件的函数；按顺序追踪程序中的数值变化。本章使用 Python 3，不使用 NumPy 或机器学习库。配套实验只依赖标准库。模块、文件读写、异常的完整机制、对象引用与调试流程在第 3 章继续；本章提前使用的少量输入检查会给出局部解释，不以此替代后续系统讨论。

机器学习实验把输入表示为程序对象，将公式中的逐项运算写为表达式，再利用循环或库函数处理许多样本。程序出现错误时，需要能够判断问题出在数据、控制流程还是计算规则。本章先建立这种基本阅读与编写能力，随后才进入数组和向量化。

程序的核心不是记住所有语法，而是对每一步的状态作出明确判断：这一行读取哪些值、产生什么值、修改了什么对象、下一行从哪里继续。为此，本章使用小规模且可手算的数据，并在重要代码后提供逐行分析。

2.1 运行程序：解释器、终端与编辑器

三个工具的职责

解释器 (interpreter) 执行 Python 程序。终端 (terminal) 提供输入系统命令的窗口，窗口内通常由命令行 **Shell** 解析命令。编辑器 (editor) 用于创建、保存和修改文本文件。集成开发环境可以把编辑、终端和运行按钮放在同一个窗口，但这些职责仍然不同。

终端里输入 `python3 hello.py`，Shell 启动 Python 解释器，再让解释器执行 `hello.py` 中的代码。Python 交互环境里输入 `2 + 3`，则是已经启动的解释器直接计算表达式。若把 `python3 hello.py` 写进 Python 文件，它不是一条合法 Python 语句；若在普通终端直接输入 `print(2 + 3)`，也不等于让 Python 运行这段代码。

位置	输入内容示例	实际负责者
普通终端	<code>python3 hello.py</code>	Shell 启动解释器
Python 交互环境	<code>2 + 3</code>	解释器求值
文本编辑器	<code>print(2 + 3)</code>	保存代码，运行时才执行
程序输出区域	数字 5	显示结果，不是新代码

本章代码框中的行号仅用于说明，不属于程序文本。终端命令框不含行号。将代码复制到文件时，不要加入解释器的提示符 `>>>`，也不要复制输出结果。

检查解释器与第一次交互

在 macOS 或 Linux 终端中运行

```
python3 --version
```

它应打印 Python 3 的版本信息。Windows 可尝试 `py -3 --version`；若系统将命令注册为 `python`，也可使用 `python --version`，但需确认确实是 Python 3。若命令不存在，说明解释器未安装、启动器未配置或路径未设置，应通过 Python 官方安装说明处理，不能把未找到命令误认作程序语法错误。

随后运行 `python3`，进入交互环境。下列框为输入与输出的完整示意：

```
>>> 2 + 3
5
>>> value = 8
>>> value * 2
16
>>> print(value)
8
>>> exit()
```

`>>>` 是解释器显示的提示符。赋值行 `value = 8` 通常不自动打印值，表达式 `value * 2` 在交互环境会显示求值结果。`exit()` 离开交互环境，返回普通终端。此后才输入执行脚本的系统命令。

保存与运行脚本

脚本（script）是包含程序文本的文件。创建 UTF-8 编码的纯文本文件 `hello.py`，保存如下内容：

```
1 name = "Python"
2 count = 3
3 print("language:", name)
4 print("count:", count)
5 print(2 + count)
```

在文件所在目录打开终端，运行 `python3 hello.py`，输出应为

```
language: Python
count: 3
5
```

解释器按顺序执行：先把名称 `name` 绑定到字符串，把 `count` 绑定到整数，再进行三次打印。`print` 括号中的多个项默认用空格分隔，打印后换行。

脚本中的单独表达式 `2 + count` 会被计算，但通常不显示结果。若要看到它，必须写 `print(2 + count)`。这是交互环境和脚本在初学阶段最容易混淆的区别之一。

目录、路径与空格

终端有当前工作目录。相对文件名 `hello.py` 会在当前目录中寻找，不会自动在整个电脑上搜索。macOS 或 Linux 使用 `pwd` 查看当前目录，`ls` 查看文件；Windows 命令提示符可用 `cd` 显示当前目录、`dir` 列出文件。

本讲义配套实验位于含中文和空格的目录，可在 macOS 终端中运行：

```
cd "/Users/mac/Desktop/机器学习 ML/配套实验"
python3 ch02_basics.py
```

路径两侧的引号让 Shell 把它看成一个整体。这里的引号用于系统命令中的路径，不是 Python 文件中的字符串定义。也可以使用脚本的绝对路径运行，不必先切换目录；但更复杂的程序可能读取相对文件，届时工作目录仍需核对。

保存时应核对扩展名。正确名称为 `hello.py`；若名称为 `hello.py.txt`，实际仍是文本扩展名。代码中的括号、冒号、引号和逗号使用英文半角符号。中文全角冒号或智能引号看起来相近，却不属于相同的语法字符。

2.2 虚拟环境、标准库与依赖

为什么区分环境

标准库随 Python 提供，包含数学、随机数、文件路径等许多功能。第三方包是另外安装的代码，例如后续使用的 NumPy、pandas 和 PyTorch。程序所依赖的包及其版本称为依赖（dependency）。本章四个实验无需安装第三方包。

不同项目可能要求不同包版本。如果把所有包混装在一个环境中，一次升级可能影响另一项目。虚拟环境（virtual environment）为一个项目隔离包安装位置，并提供与该环境关联的解释器入口。它不是虚拟机，不会另建一套完整操作系统。

在项目目录中创建环境：

```
python3 -m venv .venv
```

`-m venv` 表示让选定的解释器运行标准库中的 `venv` 模块；`.venv` 是环境目录名。创建成功后，再根据 Shell 激活：

终端类型	激活命令
macOS/Linux 的常见 Shell	<code>source .venv/bin/activate</code>
Windows 命令提示符	<code>.venv\Scripts\activate.bat</code>
Windows PowerShell	<code>.venv\Scripts\Activate.ps1</code>

激活后通常使用 `python` 调用环境解释器，退出可用 `deactivate`。PowerShell 可能受脚本执行策略限制；可按官方说明处理，或直接调用环境解释器，无须为运行本章实验修改系统级策略。

安装位置与解释器对应

无需激活也可以明确调用环境中的解释器，macOS/Linux 为 `.venv/bin/python`，Windows 为 `.venv\Scripts\python.exe`。这种方式减少“包安装到了一个环境，脚本却在另一个环境运行”的混淆。

需要安装第三方包时，优先使用 `python -m pip`，例如后续章节的环境中可能运行 `python -m pip install numpy`。这会使用当前指定解释器对应的包管理工具。安装不是本章的实验步骤，提前安装机器学习库也不能代替掌握基础语法。

编辑器可能另有解释器选择设置。即使终端激活了环境，编辑器运行按钮也不必自动采用相同解释器。可用下列小程序查看实际执行者：

```
1 import sys
2 print(sys.executable)
3 print(sys.version)
```

`import sys` 导入标准库模块，下一行打印解释器路径，再下一行打印版本。模块的一般组织方式将在第 3 章展开。此处只把这段程序作为环境核对工具。

注意：环境准备的最小要求

能确认 Python 3 解释器、保存 UTF-8 脚本并在正确目录运行，就能完成本章实验。虚拟环境为后续依赖管理建立习惯；它不会改变 `if`、`for`、列表或函数的基本语义。

2.3 变量、赋值、对象与表达式

名称绑定与顺序执行

变量在本章可理解为指向某个值的名称。更准确地说，赋值建立名称与对象的绑定。写 `weight = 2.5` 时，右边表达式先求值，再把左边名称绑定到结果。`=` 不是数学中两个表达式永远相等的声明。

```
1 x = 3
2 x = x + 2
3 y = 2 * x
4 print(x, y)
```

第一行后 `x` 为 3；第二行先读取旧 `x`，算出 5，再更新绑定；第三行读取当前 `x`，得到 `y` 为 10。最终输出 `5 10`。不存在代数方程 $x = x + 2$ 所导致的矛盾，因为它描述了两个不同执行时刻。

如果名称还未赋值就被读取，例如脚本第一行直接写 `print(weight)`，会产生 `NameError`。给名称赋值时不需要先声明固定类型，但对象具有类型。`count = 3` 后又写 `count = "three"` 是允许的；后续代码若期待数字，仍可能产生类型错误。因此名称应保持稳定含义，避免随意切换角色。

命名与注释

名称可以包含字母、数字和下划线，不能以数字开头，不能使用 `for`、`if` 等关键字。Python 允许部分 Unicode 名称，本讲义代码采用清晰的英文名称与下划线，例如 `train_size`、`predicted_cost`。大小写不同表示不同名称：`Score` 和 `score` 不相同。

不要将 `sum`、`list`、`str`、`min` 等常用内置功能名称用作变量。若写 `sum = 10`，再写 `sum(values)`，名称已经指向整数，便不能作为函数调用。变量名称应表达意义，而不是仅方便输入。

`#` 后直到行末为注释，解释器不执行这些文字。注释应解释规则、单位或约束，而不是重复显而易见的代码。如下写法记录了单价单位：

```
1 price_per_kg = 2.0 # unit: yuan per kg
2 weight_kg = 3.5
3 cost_yuan = price_per_kg * weight_kg
```

字符串中的 `#` 只是普通字符，例如 `"order#17"`。长说明可以放在文档字符串中，函数一节会给出示例。

表达式、语句与增强赋值

表达式求出一个值，例如 `2 * x + 1`、`score >= 60` 或 `len(values)`。语句执行某种动作或控制流程，例如赋值、条件分支和循环。函数调用既可以在表达式中提供返回值，也可以作为独立语句执行。

`total += value` 在数值累加场景中相当于把 `total + value` 的结果重新赋给 `total`。它不能省略初始化：第一次累加前要给 `total` 赋初值。容器的增强赋值可能涉及对象修改，引用与副作用的完整区别将在第 3 章讨论，本章主要使用数值累加。

多重赋值如 `a, b = 2, 5` 先求右边，再绑定左右对应名称；`a, b = b, a` 因此能交换数值。若分两行 `a = b`、`b = a`，第一行已经覆盖旧 `a`，第二行将得到与 `a` 相同的值。

例题 2.1：赋值追踪与交换

执行下列代码后，`a`、`b` 和 `c` 分别是多少？为什么？

```
1 a = 4
2 b = a + 3
3 a = 2 * a
4 c = b - a
5 a, b = b, a
6 print(a, b, c)
```

解 第 1 行建立 `a=4`。第 2 行得到 `b=7`。第 3 行把 `a` 改为 8，不会让先前计算好的 `b` 重新计算。第 4 行得到 `c=-1`。第 5 行同时使用旧 `b=7` 和旧 `a=8`，交换后 `a=7, b=8`。输出为 `7 8 -1`。变量不是自动维护的电子表格公式，修改输入名称不会回溯执行旧语句。

2.4 数值类型、运算与浮点误差

整数、浮点数与类型转换

整数类型为 `int`，例如 3、0 和 -7；浮点数类型为 `float`，例如 3.0、0.25 和 1e-3。科学计数形式 1e-3 表示 10^{-3} 。Python 整数可以随数值增长使用更多存储空间，不等于机器学习数组中的固定宽度整数；浮点数精度则有限。

用 `type(value)` 查看类型。`int(3.9)` 得到 3，它朝零截断，并不是四舍五入；`int(-3.9)` 得到 -3。`float("3.5")` 把合法数字字符串转为浮点数，`int("3.5")` 会失败，因为该字符串不是整数格式。`str(3.5)` 则产生文本"3.5"，不再是可直接进行数值加法的浮点数。

类型转换需要有数据规则。用户输入"abc" 不能直接转为浮点数；空串也不能。字符串中有数字并不保证整体是合法数字，例如"score=82"。完整输入校验与异常处理在第 3 章学习，本章实验采用范围有限且能事先辨认的整数文本。

运算符与优先级

符号	含义	示例与结果
<code>+ - *</code>	加、减、乘	<code>3 * 4</code> 为 12
<code>/</code>	真除法	<code>7 / 2</code> 为 3.5
<code>//</code>	向下取整除法	<code>7 // 2</code> 为 3
<code>%</code>	余数	<code>7 % 2</code> 为 1
<code>**</code>	幂	<code>2 ** 3</code> 为 8
<code>abs(x)</code>	绝对值	<code>abs(-2)</code> 为 2

加减低于乘除，幂的优先级较高；应优先通过括号明确数学意义。均方误差的一项应写 `(prediction - target) ** 2`，若写 `prediction - target ** 2`，只会把真实值平方再相减。

负数除法尤其需要核对：`-7 // 2` 为 -4，因为 -3.5 向下取整到 -4，不是朝零截断到 -3。配套余数`-7 % 2` 为 1，仍满足

$$a = (a // b) b + (a \% b), \quad b \neq 0.$$

若 *a, b* 都为整数，则这些算术规则可用于整除与余数判断。写 `i % 2 == 0` 判断整数是否偶数。除数为零会报错，不能将“没有样本”的平均值直接计算为 `total / 0`。

`-2 ** 2` 得到 -4，解析为`-(2 ** 2)`；`(-2) ** 2` 得到 4。多层幂如 `2 ** 3 ** 2` 按 `2 ** (3 ** 2)` 解释。复杂表达式宜用括号和中间名称分解，减少依赖记忆优先级。

例题 2.2: 把数学式译为程序

给定预测 $\hat{y} = 9$ 、真实值 $y = 6$ ，计算绝对误差、平方误差，以及 $\sqrt{\frac{1+4+9}{3}}$ 。写出不会混淆优先级的代码。

```
1 prediction = 9
2 actual = 6
3 absolute_error = abs(prediction - actual)
4 squared_error = (prediction - actual) ** 2
5 root_mean_square = ((1 + 4 + 9) / 3) ** 0.5
6 print(absolutely_error, squared_error, root_mean_square)
```

解 绝对误差为 3，平方误差为 9，最后一项为 $\sqrt{14/3} \approx 2.160246899$ 。第 5 行先求和、再平均、最后开方，括号直接对应公式结构。指数 0.5 在非负实数上用于平方根；这里被开方量非负，条件满足。

浮点数不是所有小数的精确表示

常见 Python 浮点数使用有限位二进制表示。十进制有限小数不一定有有限二进制展开，类似 $1/3$ 在十进制中必须近似。因此

```
1 print(0.1 + 0.2)
2 print(0.1 + 0.2 == 0.3)
```

通常输出 `0.30000000000000004` 和 `False`。这不是加法规则改变，而是输入数值和运算结果以邻近的可表示数近似。

`round(value, 2)` 和格式化保留两位小数可以改善显示，但不能让任意内部数值变成精确十进制数。`round` 还采用特定的舍入规则，对恰好中点的可精确表示数倾向于保留偶数，例如 `round(2.5)` 为 2、`round(3.5)` 为 4。`round(2.675, 2)` 可能为 2.67，因为 2.675 本身已被二进制近似，不宜仅用十进制手工舍入直觉推断。

对计算结果检查，应采用容差：

```
1 import math
2 value = 0.1 + 0.2
3 print(math.isclose(value, 0.3, rel_tol=1e-9, abs_tol=1e-12))
```

输出 `True`。`rel_tol` 随数值规模设定相对容差，`abs_tol` 为接近零的比较提供绝对容差。大致判断条件为

$$|a - b| \leq \max(r \max(|a|, |b|), \varepsilon),$$

其中 r 为相对容差， ε 为绝对容差。具体容差应依目标单位、计算误差和问题要求设定，而不是所有问题都固定使用同一值。

注意：误差来源需要区分

输入测量误差、有限样本造成的统计波动、模型预测误差和计算机浮点舍入误差，是不同来源。显示更多小数不能减少前几种误差。对费用、成绩等问题，首先保持单位与计算规则一致，再讨论数值精度。

2.5 字符串、输入与输出

字符串及其索引

字符串（string, str）表示文本，使用单引号或双引号包围，例如"train"。引号不是字符串内容的一部分。"12" + "3" 得到"123"，12 + 3 得到 15；"12" + 3 因类型不兼容而失败。

字符串可拼接或重复："ab" * 3 得到"ababab"。len(text) 给出字符串长度；text[0] 读取第一个字符；text[-1] 读取最后一个字符。Python 索引从 0 开始，长度为 n 时，正常非负索引为 0 至 $n - 1$ 。空字符串没有第一个或最后一个字符，取相应索引会报错。

```
1 word = "Python"
2 print(len(word))
3 print(word[0], word[-1])
4 print(word[:2], word[2:])
```

输出依次为 6、P n、Py thon。切片语法会在容器一节系统解释。对普通中文字符串，len(" 机器学习") 为 4，但 Unicode 字符串长度一般不等于用户视觉上感知的字符数量，带组合符号的文字可能需要更细处理。本章数据不涉及这些复杂情形。

常用文本处理与不可变性

text.strip() 返回去掉两端空白的新字符串，text.lower() 返回小写形式，text.split(",") 按逗号拆分，",".join(parts) 将多个字符串用逗号连接。点号形式表示访问对象提供的方法（method）；类与对象的系统讲解在第 3 章进行，本章先按“对这段文本执行某种操作”理解。

```
1 raw = "  Train,VALID,test  "
2 clean = raw.strip().lower()
3 parts = clean.split(",")
4 print(raw)
5 print(clean)
6 print(parts)
7 print(" | ".join(parts))
```

raw 仍含原来两端的空格，clean 为"train,valid,test"，parts 为三个字符串组成的列表，最后输出 train | valid | test。方法可以连续调用，是因为 strip() 返回的仍是字符串，接着可调用 lower()。

字符串是不可变对象，不能写 word[0] = "p" 修改单个字符。若需要改变，可构

造新字符串"p" + word[1:] 并重新绑定名称。列表则允许逐项修改，二者不能混用。

格式化、转义与交互输入

格式化字符串 `f"MAE={mae:.3f}"` 将变量 `mae` 的值嵌入文本，并按三位小数显示。这里的 `mae` 应是数值变量。百分比格式 `f"rate={rate:.1%}"` 把比例 0.8 显示为 80.0%，不改变原变量。

```
1 score = 82
2 rate = 0.8
3 print(f"score={score}, rate={rate:.1%}")
```

输出 `score=82, rate=80.0%`。第 1 章脚本使用 `"...".format(...)`，也是格式化方法，采用位置参数替代占位符。两种形式都可用，初学阶段选一种并保持一致即可。

`\n` 表示换行，`\t` 表示制表符，字符串中的反斜杠需按转义规则处理。写 Windows 路径时可使用原始字符串如 `r"C:\data\input.txt"`，但一般文件路径操作会在第 3 章采用更稳妥的路径工具。

`input("weight: ")` 显示提示并读取一行输入，返回字符串，即使输入的是 2.5。若任务要求数值，应明确转换：

```
1 text = input("weight in kg: ")
2 weight = float(text)
3 print(4 + 2 * weight)
```

输入 2.5 时输出 9.0。非法文本会在转换时失败；本章自动运行的实验使用固定数据而不是等待输入，方便重复核对。

2.6 布尔值、比较与条件分支

比较产生真假值

布尔值 (boolean) 只有 `True` 和 `False`，注意首字母大写。比较运算包括 `==`、`!=`、`<`、`<=`、`>` 和 `>=`。`=` 是赋值，`==` 才是比较两个值是否相等。

`and` 表示两个条件均满足，`or` 表示至少一个满足，`not` 否定条件。对布尔条件 `score >= 0 and score <= 100`，也可以写链式比较 `0 <= score <= 100`。数学中的区间条件与程序表达因此直接对应。

Python 使用短路求值：`A and B` 在 `A` 为假时不必求 `B`，`A or B` 在 `A` 为真时不必求 `B`。例如 `len(values) > 0 and values[0] > 0`，空列表时前半为假，不会访问不存在的第一个元素。若顺序交换，则可能先触发索引错误。

`and` 和 `or` 一般返回被选中的操作数，不一定返回布尔值。例如 `0 or 5` 为 5，`3 and 7` 为 7。在本章的条件表达中主要使用真正的比较，避免依赖这种更宽泛语义。数字 0、空字符串、空列表和 `None` 等被视为假，但“假”不等于“无效数据”：成绩 0 是合法数值，实验将专门保留它。

缩进、冒号与分支结构

```
1 score = 76
2 if score >= 90:
3     grade = "A"
4 elif score >= 60:
5     grade = "pass"
6 else:
7     grade = "fail"
8 print(grade)
```

`if` 从第一个条件开始；不满足时才检查 `elif`；前面的条件全不满足才执行 `else`。每个分支标题末尾有冒号，分支内部缩进四个空格。同一层级的缩进必须一致，避免混用制表符与空格。

当分数为 76 时，第一个条件为假，第二个为真，执行 `grade = "pass"`，不再执行其他分支。最后 `print` 没有缩进，属于分支结构之后，无论哪个分支被选中都会运行。

三个独立 `if` 与一个 `if/elif/else` 结构不同。独立 `if` 会逐个检查，可能执行多个代码块。如果先判断 `score >= 60`，再用 `elif score >= 90`，高分样本会被第一分支提前处理，后面的 90 分条件永远无机会成立。

例题 2.3：边界和条件顺序

将非负重量按收费规则处理：重量不超过 1 kg 收 6 元；超过 1 kg 但不超过 3 kg 收 10 元；超过 3 kg 收 15 元。负重量应拒绝。写出完整分支并核对 0、1、3 和 3.1 kg。

```
1 weight = 3.0
2 if weight < 0:
3     cost = None
4 elif weight <= 1:
5     cost = 6
6 elif weight <= 3:
7     cost = 10
8 else:
9     cost = 15
10 print(cost)
```

解 四个核对输入的结果分别为 6、6、10、15。`None` 是表示“没有普通结果”的特殊对象，不是字符串 `"None"`，也不是数值 0。当前教学规则允许 0 kg，真实收费规则是否允许需要另行定义。前面分支已经排除 `weight <= 1`，到第 6 行时无需再重复 `weight > 1`。

输入若为字符串而非数值，比较仍可能失败；若为某些特殊浮点数，也需要额外校验。此例的假设是输入为普通有限数值，只负责范围和边界规则，不宣称是完整的生产级输入处理。

2.7 列表、元组与切片

列表与逐项访问

列表 (list) 保存按顺序排列的一组对象, 使用方括号, 例如 `scores = [82, 91, 76]`。顺序属于数据含义的一部分, `scores[0]` 为 82, `scores[-1]` 为 76, `len(scores)` 为 3。 `scores[3]` 越界, 因为最大非负索引是 2。

列表可以混合不同类型, 但用于数值实验时应尽量保持含义和类型一致。 `[1, "2", None]` 不能直接当成三个数值平均; 需要先说明文本转换和缺失处理。空列表写作 `[]`, 它有长度 0, 不能直接取首项。

列表是可变对象, 可修改、增加和删除元素:

```
1 scores = [82, 91, 76]
2 scores[1] = 93
3 scores.append(88)
4 last = scores.pop()
5 print(scores)
6 print(last)
```

第 2 行将第二项替换为 93; 第 3 行把 88 加到末尾; 第 4 行删除并返回末尾 88, 所以最后列表为 `[82, 93, 76]`, `last` 为 88。 `append` 是在原列表上修改, 并返回 `None`。若写 `scores = scores.append(88)`, 会把名称 `scores` 绑定到 `None`, 这是常见错误。

`extend` 逐项加入另一个可迭代对象, `append` 则将参数作为一个整体加入:

```
1 a = [1, 2]
2 a.append([3, 4])
3 b = [1, 2]
4 b.extend([3, 4])
5 print(a)
6 print(b)
```

输出分别是 `[1, 2, [3, 4]]` 和 `[1, 2, 3, 4]`。前者长度为 3, 第三项本身是列表; 后者长度为 4, 所有项为整数。嵌套数据读取错误往往来自没有先确定层级与形状。

切片与半开区间

切片 `values[start:stop:step]` 按指定步长选取一段序列。最常见的正步长形式包含起点、不包含终点, 即半开区间 `[start, stop)`。省略起点默认从开头, 省略终点默认到末尾, 省略步长默认为 1。

```

1 values = [10, 20, 30, 40, 50, 60]
2 print(values[1:4])
3 print(values[:3])
4 print(values[3:])
5 print(values[::-2])
6 print(values[::-1])

```

输出依次为 [20, 30, 40]、[10, 20, 30]、[40, 50, 60]、[10, 30, 50] 和 [60, 50, 40, 30, 20, 10]。[::-1] 按反向顺序选取所有项，不会修改原列表。

长度为 n 的序列在边界内可以用 `values[:r]` 和 `values[r:]` 拆成前后两段，不遗漏也不重复第 r 项。把切片误读为包含终点，就会在数据划分中多取一个样本。切片越界通常会被截到可用范围，例如 `values[:100]` 取得全部六项；单项索引越界则报错。步长不能为 0。

对普通列表，切片创建一个新列表。若原列表中的元素本身仍是可变对象，切片并不递归复制内部对象。本章一维数值列表不用展开该问题，第 3 章将通过引用和浅复制详细讨论。不要将列表切片的行为直接套用到第 4 章的 NumPy 数组，数组切片可能是视图。

例题 2.4：划分与配对不能分离

特征为 $x = [1, 2, 3, 4, 5]$ ，标签为 $y = [5, 7, 9, 11, 13]$ 。以前三条训练、后两条评价。写出切片，并说明若只把 x 倒序会发生什么。

```

1 x_train, x_eval = x[:3], x[3:]
2 y_train, y_eval = y[:3], y[3:]
3 print(x_train, y_train)
4 print(x_eval, y_eval)

```

解 训练对应关系为 (1,5), (2,7), (3,9)，评价为 (4,11), (5,13)。若只令 $x = x[::-1]$ ，标签顺序不变，就会得到 (5,5), (4,7), ...，破坏样本与标签配对。统一把数据表示为记录对，再对记录整体排序和切片，可减少此类错误。这里的前后切片仅用于解释语法，不自动意味着任意任务都应按当前行顺序划分。

元组、解包与嵌套记录

元组 (tuple) 也是有序序列，常用于固定结构的记录。(2.5, 8.6) 可以表示“重量、费用”对；不能通过 `row[0] = 3.0` 修改其中的引用。单元素元组写成 (5,)，末尾逗号不能省略，否则 (5) 只是带括号的整数。

解包 (unpacking) 将序列中的项分配给多个名称：

```

1 row = (2.5, 8.6)
2 weight, cost = row
3 print(weight, cost)

```

右边有两项，左边有两个名称，因此一一对应。数量不匹配会报错。前文的 `a`, `b`

= b, a 也使用了这种机制。

多个记录可保存为“列表中的元组”：

```
1 records = [(1.0, 6.0), (2.0, 8.0), (3.0, 10.0)]
2 print(records[1])
3 print(records[1][0])
```

第一项索引 1 取出第二笔记录 (2.0, 8.0)；随后索引 0 取出该记录的重量 2.0。读取嵌套结构时应从左到右判断每一步返回的对象。

元组的不可变性限制的是它自身各位置的引用。如果元组内部装着列表，那个列表仍可能被修改。因此不能仅因数据外层是元组，就推断内部数据深度不可变。为保持本章语义简单，实验记录对只含数值或字符串。

2.8 字典、集合与容器选择

字典按键组织数据

字典 (dict) 把键 (key) 映射到值 (value)，例如

```
1 summary = {"count": 5, "mean": 67.4, "pass_rate": 0.8}
2 print(summary["mean"])
3 summary["maximum"] = 91
4 print(summary.get("median"))
```

按键 "mean" 查到 67.4，新键 "maximum" 被加入。get 在键不存在时默认返回 None，也可写 summary.get("median", 0) 提供默认值。但是默认 0 是否合理取决于任务，统计量缺失通常不应悄悄当成 0。

普通索引 summary["median"] 在键不存在时会报 KeyError，有助于发现程序假设错误。if "mean" in summary 检查键是否存在，而不是检查 67.4 是否在所有值中。summary.keys()、summary.values() 和 summary.items() 分别用于访问键、值和键值对。

字典适合表示固定命名字段，如一份统计报告或一条订单记录。列表适合按位置遍历一批样本。实际数据也可以是记录字典组成的列表。读取时先用 records[i] 选出一个样本，再按键 "weight" 读取重量；完整写法如下。

```
records[i]["weight"]
```

Python 3 字典保留插入顺序，但“按插入顺序迭代”不等于“按键的大小排序”。重复赋值同一个键会替换旧值，不会保存两条同名键值对。

```
1 counts = {"pass": 0, "fail": 0}
2 counts["pass"] = counts["pass"] + 1
3 counts["pass"] += 1
4 print(counts)
```

最终 "pass" 对应 2，"fail" 对应 0。这类计数与第一章混淆矩阵的四个类别计数具有同样的累加结构。

集合用于去重和成员关系

集合 (set) 保存互不重复的元素，没有用于本章数据配对的确切位置顺序。例如 `set(["A", "B", "A"])` 只含 A 与 B。空集合必须写 `set()`，`{}` 是空字典。

常用运算包括交集 `a & b`、并集 `a | b` 和差集 `a - b`。若训练用户标识集合与测试用户标识集合交集非空，说明存在跨集合用户重叠；是否违反任务协议，还要看第一章中的使用目标。

```
1 train_users = {"u1", "u2", "u3"}
2 test_users = {"u3", "u4"}
3 overlap = train_users & test_users
4 print(sorted(overlap))
```

输出 `['u3']`。用 `sorted` 将集合变成稳定排序的列表，便于显示。不要用集合存放需要与标签位置一一对应的特征序列，去重和无位置语义会破坏配对。

列表、元组和字典等某些可变对象不能直接作为集合元素或字典键，因为它们不具备所需的可哈希性。字符串、整数以及由合适不可变元素构成的元组通常可以。哈希的机制在后续算法中再说明，本章只需选择清楚、合法的键类型。

容器	最直接的用途	应避免的误用
列表	有序样本与可修改序列	忘记索引从 0 开始
元组	固定结构的一条记录	误以为内部对象都不可变
字典	命名字段、键到值的映射	把键的存在当成值的存在
集合	去重、交并差、成员判断	用其位置连接特征与标签

2.9 for 循环：遍历、累加与配对

从一项公式到多项计算

for 逐项遍历可迭代对象。对列表计算平均数：

```
1 values = [6.0, 8.0, 10.0]
2 total = 0.0
3 for value in values:
4     total += value
5 mean = total / len(values)
6 print(mean)
```

循环开始前 `total=0.0`。每次取出一个列表元素，绑定到 `value`，执行缩进内部的累加。三个循环后 `total=24.0`，离开循环，再除以 3 得到 8.0。平均行若错误缩进循环，就会在尚未累加完整时不断计算中间结果。

轮次	当前 value	累加前 total	累加后 total
1	6.0	0.0	6.0
2	8.0	6.0	14.0
3	10.0	14.0	24.0

这个累加器对应数学求和 $\sum_i v_i$ 。若列表为空，循环一次也不执行，total 仍为 0.0，但平均值除以 0，因此必须另行处理空输入。

range 与边界

`range(stop)` 提供 0 到 `stop-1` 的整数序列。完整形式可指定起点、终点和步长，如下面三个示例所示。它不是立即生成一个普通列表，但可在循环中逐项使用；调用 `list` 可显式转成列表。

```
1 print(list(range(4)))
2 print(list(range(1, 5)))
3 print(list(range(5, 0, -2)))
```

输出为 `[0, 1, 2, 3]`、`[1, 2, 3, 4]`、`[5, 3, 1]`。终点仍被排除，反向步长时只要继续满足方向边界就取下一项。

计算 1 至 n 的和可写 `for i in range(1, n + 1)`。若误写 `range(1, n)`，会漏掉 n 。不需要位置时，应直接 `for value in values`，减少不必要索引；需要下标时可以使用 `enumerate`。

enumerate 与 zip

`enumerate(values)` 逐项产生“下标、值”对：

```
1 values = [6, 8, 10]
2 for index, value in enumerate(values):
3     print(index, value)
```

输出三行为 `0 6`、`1 8`、`2 10`。它适合错误定位和逐条报告，不需要手动维护另一个下标变量。

`zip(actual, predicted)` 把相同位置的两个值配成一对：

```
1 actual = [6, 8, 10]
2 predicted = [5, 9, 10]
3 total = 0.0
4 for y, y_hat in zip(actual, predicted):
5     total += abs(y_hat - y)
6 print(total / len(actual))
```

绝对误差为 1、1、0，结果为 2/3。循环直接对应 $\sum_i |\hat{y}_i - y_i|$ ， y 和 y_hat 的名称与公式角色一致。

默认 `zip` 在最短输入结束时停止。若真实值有 3 项、预测只有 2 项，最后一项会被遗漏，程序可能仍给出一个数。这是静默逻辑错误，不一定报错。指标函数应先检查两者等长且非空。本章采用显式长度检查，以保持对常见 Python 3 环境的兼容。

例题 2.5：错误代码为什么看似能运行

真实值为 `[2, 4, 6]`，预测为 `[3, 5]`。某程序使用 `zip` 累加绝对误差，再除以 `len(actual)`。它输出什么？为什么不能称为该数据的 MAE？

解 `zip` 只产生 `(2,3)` 和 `(4,5)` 两对，误差总和为 2，再除以 3 得到 `2/3`。第三个真实值没有对应预测，问题本身缺少完整输入。分子只计算两项，分母却计算三项，既不是完整三条记录的 MAE，也不是两条有效配对的 MAE。正确处理是拒绝长度不匹配的输入，而不是通过更换分母掩盖缺失预测。

`break`、`continue` 与嵌套循环

`break` 立即结束当前最近一层循环，`continue` 跳过本轮剩余代码并进入下一轮。例如查找第一个超过 100 的值：

```
1 values = [82, 91, 105, 88]
2 first_bad = None
3 for index, value in enumerate(values):
4     if value > 100:
5         first_bad = index
6         break
7 print(first_bad)
```

第 3 项 105 在下标 2 出现，记录 2 后结束循环，不检查 88。初始化 `None` 保证即使没有错误，变量仍有明确结果。

忽略负数但保留 0 可以写：

```
1 clean = []
2 for value in [3, -1, 0, 5]:
3     if value < 0:
4         continue
5     clean.append(value)
6 print(clean)
```

输出 `[3, 0, 5]`。若以 `if not value` 代替负数判断，会错误跳过 0，却无法跳过 -1。真假语义不等于任务有效性规则。

嵌套循环适用于遍历二维结构：

```

1 rows = [[1, 2], [3, 4], [5, 6]]
2 total = 0
3 for row in rows:
4     for value in row:
5         total += value
6 print(total)

```

外层取三行，内层各取两项，累计为 21。若每行有 d 项、共 n 行，该程序进行 nd 次加法，时间随 nd 增长。理解这些循环是第 4 章向量化改写的前提。

2.10 while 循环与终止条件

`while` 在条件为真时重复执行代码块，适合重复次数尚未确定但有停止条件的过程。

```

1 value = 1
2 steps = 0
3 while value < 20:
4     value *= 2
5     steps += 1
6 print(value, steps)

```

每轮开始先检查 `value < 20`，成立才执行。`value` 依次变成 2、4、8、16、32，`steps` 为 5。得到 32 后下次检查不成立，结束；条件不是执行过程中时时检查，所以结果可能越过边界。

轮次	检查前 value	条件	执行后 value
1	1	真	2
2	2	真	4
3	4	真	8
4	8	真	16
5	16	真	32
下一次	32	假	不执行

若漏写 `value *= 2`，条件永远为真，就产生无限循环。写 `while` 前应回答：哪些变量决定条件；它们每次怎样改变；在什么输入条件下能停止。迭代算法通常还应有最大步数，防止数值条件永远达不到。

一个简单的收敛示意图为不断将差距减半：

```
1 gap = 1.0
2 steps = 0
3 while gap > 0.01 and steps < 20:
4     gap /= 2
5     steps += 1
6 print(gap, steps)
```

7 步后 gap 为 0.0078125，满足容差而结束。最大 20 步是保护条件。该例只演示控制流程，不是某个机器学习算法的完整收敛证明。

当重复次数已知、对象可直接遍历时，优先用 for；当下一次是否执行依赖动态状态时，用 while 更自然。两者都需明确初始化和终止，循环本身不会判断任务是否已经正确完成。

2.11 函数、参数、返回值与作用域

把重复计算封装为函数

函数（function）为一段有名称、可调用的计算规则。定义采用 def：

```
1 def predict_cost(weight, base=4.0, rate=2.0):
2     """Return the predicted cost in yuan."""
3     cost = base + rate * weight
4     return cost
5
6 print(predict_cost(2.5))
7 print(predict_cost(2.5, base=5.0, rate=3.0))
```

定义函数时并不立即执行内部收费计算，而是创建函数对象并绑定名称。第 6 行调用才把 weight 设为 2.5，未提供的 base、rate 采用默认值，返回 9.0。第 7 行使用关键字参数覆盖默认值，返回 12.5。

定义中的名称称为形参（parameter），调用中提供的具体对象称为实参（argument）。这里语法中的“参数”与第一章的“模型参数”不是同一个分类问题：函数参数可以传入数据、配置、模型系数甚至另一个函数；它是否属于模型可训练参数，要由学习流程决定。

首行说明字符串称为文档字符串（docstring），可以交代单位、返回值和条件。一个函数的说明应让调用者知道怎样提供输入、如何解释输出以及哪些输入不接受。

return 与 print 有不同作用

return 把结果交还给调用位置，并立即结束当前函数。print 只将内容显示到输出区域，通常返回 None。

```
1 def square_show(x):
2     print(x ** 2)
3
4
5 def square_return(x):
6     return x ** 2
7
8 shown = square_show(3)
9 returned = square_return(3)
10 print(shown, returned)
```

第8行先打印9，然后因函数没有 `return` 而把 `shown` 设为 `None`。第9行不自动打印，`returned` 获得9。最后输出 `None 9`。若后续还要平均、比较或保存结果，就应返回数据，而不是只打印。

函数可以返回多个量，实际常为一个元组：

```
1 def limits(values):
2     return min(values), max(values)
3
4 low, high = limits([3, 8, 5])
5 print(low, high)
```

返回 `(3, 8)`，调用者通过解包获得两个值。当前函数要求输入非空，否则 `min`、`max` 不能定义结果，后面会加边界检查。

位置参数、关键字参数与默认值

位置参数按顺序匹配，`predict_cost(2.5, 5.0, 3.0)` 等于把重量、起步价、单价分别传入。关键字参数通过名称匹配，适合有多个含义相近的数值时使用。常规调用中位置参数要先于关键字参数，不能把同一个形参重复赋值。

默认值在函数定义时求值。数值、字符串和 `None` 常可安全作为默认配置；可变列表默认值会被多次调用共享，容易引发累积：

```
1 # Avoid this pattern for a fresh list per call.
2 def collect_bad(value, bucket=[]):
3     bucket.append(value)
4     return bucket
```

连续调用 `collect_bad(1)`、`collect_bad(2)` 后，两次都使用同一个默认列表，第二次结果为 `[1, 2]`。若每次需要新列表，可使用

```
1 def collect(value, bucket=None):
2     if bucket is None:
3         bucket = []
4     bucket.append(value)
5     return bucket
```

`is None` 检查是否为那个特殊对象，比把 `None` 混入普通数值比较更清楚。若调用者主动传入列表，这个函数会修改该列表；这应当记录在接口说明中。引用与副作用的深入分析放在第 3 章。

局部名称与作用域

作用域 (scope) 规定名称在什么位置可见。函数内的参数和通常赋值的名称属于该次调用的局部作用域。相同名称在函数内外可以代表不同绑定：

```
1 x = 10
2
3 def add_one(x):
4     result = x + 1
5     return result
6
7 y = add_one(3)
8 print(x, y)
```

函数内部的 `x` 为 3，返回 4；外部 `x` 仍为 10，输出 `10 4`。局部 `result` 不能在函数之外直接读取。

尽量把函数所需数据作为参数传入，而不是依赖外部可变状态。若函数在内部写 `threshold`，但它既非参数也非局部定义，就要向外寻找名称，程序行为会受外部环境影响。纯计算函数如 MAE 可以只由输入决定结果，便于手算和重复核对。

赋值局部名称和修改传入对象不同。例如 `values = []` 只是使局部名称改指向新列表，`values.append(3)` 却修改了调用者可能持有的原列表。本章指标函数不修改输入，数据清理函数创建新列表，这使实验更容易追踪。

例题 2.6：建立可靠的均值函数

设计 `average(values)`，对非空数值列表返回平均值，空列表返回 `None`。用三个输入核对，并解释分支为什么放在除法之前。

```
1 def average(values):
2     if not values:
3         return None
4     total = 0.0
5     for value in values:
6         total += value
7     return total / len(values)
8
9 print(average([2, 4, 6]))
10 print(average([0]))
11 print(average([]))
```

解 输出 4.0、0.0、None。`if not values` 检查列表是否为空，`[0]` 仍是非空列表，不会误判成空输入。空输入提前返回，避免除以零，也明确表示平均值未定义。该函数

假定非空输入每项为可进行数值加法的数，不负责解析任意文本。

2.12 列表推导式、排序与函数作为输入

从普通循环到列表推导式

列表推导式 (list comprehension) 把逐项变换和筛选写成紧凑形式。先看循环：

```
1 squares = []
2 for value in [1, 2, 3, 4]:
3     squares.append(value ** 2)
```

等价写法为

```
1 squares = [value ** 2 for value in [1, 2, 3, 4]]
```

结果是 [1, 4, 9, 16]。阅读时先看 for 从哪里取值，再看前面的表达式怎样变换。加筛选可写 [v ** 2 for v in values if v >= 0]，含义是先保留非负项，再平方。

["pass" if v >= 60 else "fail" for v in values] 则对每一项选择一种输出，不丢弃元素。这里的条件表达式位于 for 前；位于末尾的 if 是过滤。两者结果长度可能不同，数据配对时不能混淆。

复杂的多层条件或需要打印中间过程时，用普通循环更容易阅读和排错。推导式减少行数并不自动提高算法效率，也不会把 Python 循环变成 NumPy 向量化。

生成器表达式与 sum

sum(values) 计算数值和；sum(v ** 2 for v in values) 在求和时逐项产生平方，不必先保存全部平方列表。括号中的形式称为生成器表达式，这里可以理解为向 sum 按需提供各项。

在 sum(v >= 60 for v in values) 中，每次比较得到布尔值，数值求和时 True 按 1、False 按 0 处理，因此得到及格人数。这是一个有意使用的计数约定，应知道它与 bool 的数值行为有关，而不是把真假值本身当成成绩。

表达式 sum(...) 遍历一次输入，时间随项数线性增长；若使用列表推导式，还要保存一份结果列表。小实验中差异很小，但理解是否创建中间列表有助于后续数据规模扩大时判断空间成本。

sorted、sort 与排序键

sorted(values) 返回新的排序列表，原对象不变；values.sort() 在原列表中排序，返回 None。这与 append 的原地修改语义一致。

```
1 values = [3, 1, 2]
2 ordered = sorted(values)
3 print(values)
4 print(ordered)
```


输出 [3, 1, 2] 和 [1, 2, 3]。若排序的是记录，需要说明根据哪个字段：

```

1 records = [(1.0, 6.0), (3.0, 10.0), (2.0, 8.0)]
2
3 def get_weight(row):
4     return row[0]
5
6 ordered = sorted(records, key=get_weight)
7 print(ordered)

```

key 接收函数，排序时用它为每条记录计算比较键。本例按重量得到 1.0、2.0、3.0 的顺序，而不是把重量和费用分别排序。

简短的 `lambda row: row[0]` 等价于此处的取重量函数，但没有独立名称。实验 1.1 中的 `lambda row: abs(row[0] - x)` 计算与查询重量的距离。Python 排序是稳定的，键相同的记录保留原来的相对次序；因此可以据此制定明确的距离同分规则。

函数作为参数与第一章代码回读

函数对象也可以作为输入。下面指标函数接收一个预测规则：

```

1 def constant_predict(x):
2     return 8.0
3
4
5 def evaluate_mae(records, predict):
6     total = 0.0
7     for x, y in records:
8         y_hat = predict(x)
9         total += abs(y_hat - y)
10    return total / len(records)
11
12 result = evaluate_mae([(1, 6), (2, 8)], constant_predict)
13 print(result)

```

第二个参数传入 `constant_predict` 函数本身，不是 `constant_predict(...)` 调用结果。指标函数每次拿到 x 后调用 `predict(x)`，因此不必知道预测规则如何实现。输出为 $(|8 - 6| + |8 - 8|)/2 = 1.0$ 。

实验 1.1 在候选循环内部使用 `lambda x: neighbor_predict(train, x, k)`，把当前 `train` 和 `k` 与预测规则结合起来，并立即执行 MAE 计算。这里没有把多个延迟执行的匿名函数存入列表，所以不会遇到以后执行时才读取循环变量的另一类问题。若后续需要长期保存多个函数，应重新检查绑定语义，第 3 章将讨论相关组织与调试。

学习完本节，可按四个问题回读第一章代码：记录是什么容器；排序键返回什么；切片取哪几项；求和与分母对应哪个数学集合。能回答这些问题，便能独立核对近邻预测和指标计算的主要逻辑。

2.13 逐行执行、错误定位与计算约定

用状态表检查程序

对每个循环记录当前输入、条件判断和累积量，能把“看懂大致意思”转为可验证的执行分析。设有成绩 [82, 59, 0, 91]，只统计达到 60 分的成绩平均值：

```

1 scores = [82, 59, 0, 91]
2 total = 0
3 count = 0
4 for score in scores:
5     if score >= 60:
6         total += score
7         count += 1
8 if count > 0:
9     mean = total / count
10 else:
11     mean = None
12 print(total, count, mean)
```

条件决定哪些样本进入分子和分母，两者必须一致。

当前成绩	是否纳入	更新后 total	更新后 count
82	是	82	1
59	否	82	1
0	否	82	1
91	是	173	2

最终输出 173 2 86.5。若分母错误使用 `len(scores)`，输出 43.25，实际上将未纳入的两条记录也计入分母。数字看起来合法，但统计意义错误。

例题 2.7：累加器初始化的位置

下面程序想计算所有平方和，却把初始化放在循环内部。预测输出，然后改正。

```

1 values = [1, 2, 3]
2 for value in values:
3     total = 0
4     total += value ** 2
5 print(total)
```

解 每次循环都先把 `total` 重置为 0，随后只加当前项，所以三轮结束时分别为 1、4、9，最终输出 9。应将 `total = 0` 移到循环之前，得到 $1 + 4 + 9 = 14$ 。如果原输入为空，错误程序还会在打印时遇到名称未定义，修正后平方和自然为 0，但这与平均数空输入未定义的规则不同。

语法、运行与逻辑错误

语法错误表示程序无法按 Python 规则解析，例如遗漏冒号、括号未配对或缩进结构不合法。运行时错误表示语法可以解析，但执行时遇到不能完成的操作，例如除以零、读取缺失键或不同类型相加。逻辑错误表示程序运行完成，却实现了错误的规则，例如把平方误差写成普通误差。

读报错时先找到最后的异常类型和说明，再看指向的文件与行号，并检查该行使用的变量。一个错误行可能由前面不正确赋值引起，例如 `scores = scores.append(88)` 本身可执行，直到后面对 `scores` 求长度才失败。需要追踪数据如何来到错误行，而不是只修改报错处。

对逻辑错误，应使用能揭示差异的小输入。检查 MAE 时，不只用全正残差，因为忘写绝对值可能偶然同样正确；应包含高估和低估。检查循环边界时，要包含恰好等于阈值的输入。检查分母时，要让被筛选数量与原数量不同。这样的核对比大量随机打印更能回答具体疑问。

接口约定与有限输入检查

函数先说明接受什么输入、返回什么结果、怎样处理不合法输入。对于 MAE 函数，本章采用约定：两个数值序列长度相同且非空；错误输入主动报 `ValueError`；不修改输入。主动报错的语法为 `raise ValueError("说明")`，它会结束正常执行并传播错误。第 3 章讨论调用者怎样捕获与处理。

```
1 def mean_absolute_error(actual, predicted):
2     if len(actual) != len(predicted):
3         raise ValueError("lengths differ")
4     if not actual:
5         raise ValueError("inputs are empty")
6     total = 0.0
7     for y, y_hat in zip(actual, predicted):
8         total += abs(y_hat - y)
9     return total / len(actual)
```

前五行在任何平均计算之前核对结构。随后 `zip` 恰好覆盖全部配对，分母与误差项数一致。它仍假定元素是合法有限数值，不是任意混合对象；更完整的数值检查要按实际任务添加。

若希望空输入返回 `None`，也可以设计另一种接口，但调用者就必须处理这一返回值。实验 2.1 的统计函数采用 `None`，实验 2.2 的指标函数采用主动拒绝，两者各自明确而一致；不能在同一函数中偶尔返回 0、偶尔返回 `None`，却不说明规则。

例题 2.8：正负误差抵消

真实值为 [5, 5]，预测为 [7, 3]。某函数直接平均 `y_hat - y` 并将结果称为 MAE。给出反例计算与修正。

解 有符号误差为 (2, -2)，平均为 0；每项预测都错了 2，MAE 应为 $(2 + 2)/2 = 2$ 。修

正累加项为 $\text{abs}(y_{\text{hat}} - y)$ 。平均有符号误差可以描述总体高估或低估倾向，但不能衡量错误大小，应该使用不同名称并解释含义。

2.14 实验 2.1：从原始文本到统计报告

任务与规则

入口为 `ch02_basics.py`。输入为九条固定文本：

```
1 raw = [" 82 ", "91", "", " 76", "absent", "88", "0", "105", "-3"]
```

任务是从文本中提取 0–100 范围内的非负整数成绩，保留原顺序，并记录每条被拒绝记录的下标、原始文本和原因。然后给出有效数量、均值、最小值、最大值与及格比例。该输入不是实际学生数据，仅演示文本处理和统计规则。

输入规则明确为：只接受由 ASCII 十进制数字组成的非空文本，可以带两端空白；不接受正负号、小数点或单位；转为整数后还需满足范围。因而“-3”在文本规则阶段被拒绝，“105”在范围阶段被拒绝，“0”被保留。拒绝原因的先后由检查顺序决定，不意味着负值没有范围问题。

清理循环与逐条追踪

```
1 clean = []
2 rejected = []
3 for index, text in enumerate(raw):
4     stripped = text.strip()
5     digits = stripped.isascii() and stripped.isdecimal()
6     if not digits:
7         rejected.append((index, text, "invalid text"))
8         continue
9     value = int(stripped)
10    if not 0 <= value <= 100:
11        rejected.append((index, text, "out of range"))
12        continue
13    clean.append(value)
```

`strip` 只去除两端空白，保留原始 `text` 用于报告。`isdecimal` 判断是否为非空十进制数字文本；再结合 `isascii` 将范围限定到本实验接受的 ASCII 数字，避免把 Unicode 数字形式混入该简化接口。通过文本检查后，`int` 转换才有明确依据。

两处 `continue` 把拒绝记录从后续处理排除；最后 `append` 只有在两次检查均通过时执行。`rejected` 保存三项元组，数据来源与原因可以逐条定位。

下标	原始文本	处理后文本	结果
0	" 82 "	"82"	保留 82
1	"91"	"91"	保留 91
2	""	""	拒绝: 空串不符合数字规则
3	" 76"	"76"	保留 76
4	"absent"	"absent"	拒绝: 不是数字文本
5	"88"	"88"	保留 88
6	"0"	"0"	保留 0
7	"105"	"105"	拒绝: 超过范围
8	"-3"	"-3"	拒绝: 含负号

有效数据为 [82, 91, 76, 88, 0], 和为 337, 平均 67.4; 及格有 4 项, 及格比例 0.8。若错误地丢弃 0, 平均变成 84.25, 及格比例变成 1.0。这会改变报告含义, 而不是仅改变程序风格。

统计函数与运行结果

```

1 def describe(values):
2     if not values:
3         return None
4     count = len(values)
5     average = sum(values) / count
6     passed = sum(value >= 60 for value in values)
7     return {
8         "count": count,
9         "mean": average,
10        "minimum": min(values),
11        "maximum": max(values),
12        "pass_rate": passed / count,
13    }

```

函数只接受已经清理的数值列表。数量、均值和及格比例共享同一个有效总体, 字段用字典命名, 方便调用者读取。空输入返回 `None`, 避免未定义统计量被伪装成正常 0 值。

运行 `python3 ch02_basics.py`, 应打印原始记录、有效列表和四条拒绝记录。统计部分为

```
count = 5
mean = 67.4
minimum = 0
maximum = 91
pass_rate = 0.8
empty input: None
```

列表、计数与拒绝下标应完全一致；小数值与手算差异应小于 10^{-9} 。完整英文错误原因见脚本和随附输出，不影响这里的规则说明。

修改任务

将"105"改成"100"，应新增一个有效值，数量 6，和 437，均值 $437/6 \approx 72.833333$ ，及格比例 $5/6$ ，最大值 100。再将"absent"改成"59"，应在原实验基础上得到数量 6、和 396、均值 66.0、及格比例 $4/6$ 。两项修改应分别从原始输入开始，避免混淆对照条件。

将原始列表改成全空串，确认统计返回 None。此时没有统计字典，不能继续遍历其字段。完整脚本针对当前非空示例打印字段；扩展到全无效输入时，应先判断 `summary is None`，再决定怎样输出。这说明函数正确处理边界后，调用程序也必须遵守它的接口。

最后把清理逻辑整理成 `clean_scores(raw)` 函数，返回 `clean`, `rejected`，保留原输入不变。习题 2.15 提供核对方案。不要只返回有效值而丢弃拒绝原因，否则报告难以追踪数据变化。

2.15 实验 2.2：预测误差与模型比较

数据、函数与数学对应

入口为 `ch02_metrics.py`。四条真实值为 (6, 8, 10, 12)，两个模型的固定预测分别为 A: (5, 9, 10, 14)，B: (6, 8, 10, 16)。实验不训练模型，只把已有预测变成可解释的评价报告。

先手算 A 的误差为 (-1, 1, 0, 2)，MAE 为 1，MSE 为 1.5，RMSE 为 $\sqrt{1.5} \approx 1.224744871$ 。B 的误差为 (0, 0, 0, 4)，MAE 同样为 1，MSE 为 4，RMSE 为 2。两个模型在 MAE 下打平，平方指标更偏好 A。

完整脚本把长度和空输入检查提取为 `validate_pairs`，两个指标函数共用。平方指标函数为

```
1 def mean_squared_error(actual, predicted):
2     validate_pairs(actual, predicted)
3     total = 0.0
4     for y, y_hat in zip(actual, predicted):
5         total += (y_hat - y) ** 2
6     return total / len(actual)
```


第 4 行取得一对标签和预测，第 5 行对应 $(\hat{y}_i - y_i)^2$ ，第 6 行对应 $1/m$ 。RMSE 不需重新遍历数据，在 MSE 结果上求平方根即可。

字典遍历、最坏误差与输出

两个模型预测保存在字典，按键分别评价：

```

1 predictions = {"A": [5.0, 9.0, 10.0, 14.0],
2               "B": [6.0, 8.0, 10.0, 16.0]}
3 for name, predicted in predictions.items():
4     errors = [p - y for y, p in zip(actual, predicted)]
5     absolute = [abs(error) for error in errors]
6     worst_index = absolute.index(max(absolute))
7     mae = mean_absolute_error(actual, predicted)
8     mse = mean_squared_error(actual, predicted)
9     print(name, mae, mse, worst_index)

```

items() 提供键值对，name 为模型名，predicted 为一组预测。max 找到最大绝对误差，index 返回第一次出现该值的下标；若有并列最大项，当前报告只记录第一项，不能宣称它是唯一最坏项。

运行 `python3 ch02_metrics.py`，主要结果为

```

model: A
errors: [-1.0, 1.0, 0.0, 2.0]
MAE=1.000000 MSE=1.500000 RMSE=1.224745
worst index: 3
model: B
errors: [0.0, 0.0, 0.0, 4.0]
MAE=1.000000 MSE=4.000000 RMSE=2.000000
worst index: 3

```

脚本还核对完全正确预测、单样本预测，并确认空输入和长度不同输入被拒绝。错误路径的捕获使用 try/except，完整语义在第 3 章展开，此处只用于让脚本能继续报告“非法输入已拒绝”。

修改任务与比较限制

把 B 最后一项 16.0 改成 14.0，则 B 误差变成 (0,0,0,2)，MAE 为 0.5，MSE 为 1，RMSE 为 1。再给两个模型加入同一条真实值 20，A 预测 21，B 预测 24，分别手算新指标。注意分母由 4 变成 5，不能只追加分子而忘记改变样本数。

删除某个模型最后一项，程序应拒绝不等长输入。为报告所有并列最坏项，可把第 6 行改为先保存 `maximum = max(absolute)`，再通过 `enumerate` 筛出所有误差等于最大值的下标。对复杂浮点计算是否采用近似并列，应另外明确容差，本例误差为简单整数无需扩大概念。

本实验中的固定预测不证明哪个算法普遍更好。指标比较只能在同一数据与评价协议下解释。若随后按本数据不断修改模型，这组数据就参与了选择；真实项目仍需

保留未参与选择的评价数据。

2.16 代码风格、知识联系与自检

基础程序采用四个空格缩进，一个名称保持一个稳定含义，变量名表达数据角色和必要单位。长表达式可以先求 `error`、再求 `absolute_error`，避免过度紧凑。函数优先返回数据，输出在调用层集中处理，便于重复使用和核对。

容器决定数据怎样组织，循环决定哪些项参与运算，函数决定接口和边界，条件决定有效性和决策规则。它们与第一章直接联系：记录对保证特征标签对齐，长度检查保证指标分母正确，切片帮助理解划分边界，字典帮助组织评价报告，排序键帮助实现近邻查询。

本章没有引入数组广播、类继承、异步程序或大型框架。下一章处理文件、模块、异常和引用，随后 NumPy 把逐项数值运算改写为数组运算。语法越简洁，越需要知道输入形状和运算对象，不能仅凭代码行数判断自己已经理解算法。

自检时应能够预测一个含分支循环程序的输出，指出终端命令与 Python 语句的区别，对空输入写出明确规则，并将一条数学求和式对应到循环中的累加项与分母。最后能在不改变数据的情况下运行四个配套实验，再逐项解释修改后的结果。

2.17 分层习题

- 2.1 运行。脚本 `hello.py` 含 `2 + 3` 和 `print(2 + 3)` 两行。运行时输出几次 5？在 Python 交互环境中能否直接输入 `python3 hello.py` 运行？说明如何在含空格目录中正确执行脚本。
- 2.2 追踪。依次执行 `a = 2`、`b = a`、`a += 3`、`b *= a`、`a, b = b, a`。写出每行后的 `a, b` 与最终输出。
- 2.3 计算。求 `7 / 2`、`7 // 2`、`-7 // 2`、`-7 % 2`、`-2 ** 2` 和 `(-2) ** 2`。为什么 `int(-3.5)` 与 `-7 // 2` 不同？
- 2.4 数值。为什么 `0.1 + 0.2 == 0.3` 可能为假？给出用于核对该计算的容差方法；能否靠格式化保留两位小数让所有后续计算精确？
- 2.5 字符串。将字符串 `"82,91,0"` 转为三个整数。若直接拆分文本后交给 `sum`，为什么失败？这里哪些文本检查可因固定输入假设省略，推广后又需补上？
- 2.6 条件。对成绩 0–100，90 及以上为 A，60–89 为 B，其余为 C，范围外为非法。写出分支，核对 `-1, 0, 59, 60, 89, 90, 100, 101`。
- 2.7 切片。给定 `v = [0, 1, 2, 3, 4, 5, 6]`，求 `v[1:6:2]`、`v[-3:]`、`v[::-1]` 和 `v[4:2]`。与 `v[20]` 相比，`v[:20]` 有什么区别？
- 2.8 容器。解释 `[1, 2].append([3, 4])` 与 `extend` 对原列表结构的不同影响；为什么不能把 `append` 返回值赋回原列表名？空字典与空集合分别怎样写？
- 2.9 循环。不使用 `sum`，用循环计算 1 至 n 的整数和。规定 $n \geq 0$ ，核对 0、1、5；说明为什么终点要写 $n + 1$ 。

- 2.10 while。**从 $x = 1$ 开始，每轮令 $x = 2 * x + 1$ ，直到 $x \geq 20$ ，统计轮数。给出代码和每轮状态。若输入上界是 1，循环是否执行？
- 2.11 字典与集合。**对 `labels = ["cat", "dog", "cat", "bird", "dog", "cat"]`，用字典统计频数，不预设类别列表。再给出不同标签集合，并说明为什么集合不能保存原位置。
- 2.12 函数。**编写成绩分类函数 `classify`，参数为成绩和默认值 60 的阈值。合法成绩按阈值返回 "pass" 或 "fail"，范围外返回 `None`。比较成绩 70 在默认阈值与阈值 80 下的结果。解释 `return` 与 `print` 的区别。
- 2.13 指标。**编写等长非空数值输入上的 MAE 与 MSE 函数，不修改输入。核对真实值 `[1, 3, 5]`、预测 `[2, 1, 5]`，并验证空输入和长度不同被拒绝。
- 2.14 推导式。**将非负项筛选后平方的循环改写成列表推导式。给定 `[-2, 0, 3, -1, 4]`，给出结果。再用一个条件表达式将每项标记为 "valid" 或 "invalid"，说明两种结果长度不同的原因。
- 2.15 实验整理。**把实验 2.1 清理部分封装成 `clean_scores(raw)`，返回有效值和拒绝记录，不修改输入。编写调用代码，在全无效输入时显示 "no valid scores"，否则打印数量与均值。
- 2.16 综合。**训练记录为 `[(1, 5), (2, 7), (4, 11)]`，评价记录为 `[(1.2, 5.5), (3.5, 10)]`。编写只使用训练记录的最近一个邻居预测函数和 MAE 评价，给出预测与误差。实现应检查训练非空；按稳定排序处理距离同分。

2.18 习题参考答案

2.1–2.4：运行、赋值与数值

2.1 只输出一次 5，第一行的表达式没有 `print`，在脚本中不自动显示。交互环境中的 `python3 hello.py` 不是 Python 语句，应先 `exit()` 返回普通终端，再在文件所在目录执行。含空格目录用引号包围，如 `cd "/path/with space"`，也可以给脚本提供带引号的绝对路径。若编辑器运行按钮使用另一解释器，要核对实际环境。

2.2 第 1 行 $a = 2$ ， b 尚未定义；第 2 行 $a = 2, b = 2$ ；第 3 行 $a = 5, b = 2$ ；第 4 行 $a = 5, b = 10$ ；交换后 $a = 10, b = 5$ 。整数绑定彼此独立，修改 a 不会自动改写 b 的过去结果。最终打印 10 5。

2.3 结果依次为 3.5、3、-4、1、-4、4。向下取整除法将 -3.5 变成 -4，`int(-3.5)` 朝零截断为 -3。幂先于无括号的一元负号，故 `-2 ** 2` 是 $-(2^2)$ ；给底数加括号后才是 $(-2)^2$ 。

2.4 常见浮点表示不能精确存储 0.1 和 0.2，运算后与 0.3 的存储近似值有微小差异。可以 `math.isclose(0.1 + 0.2, 0.3, rel_tol=1e-9, abs_tol=1e-12)` 核对。格式化只生成显示字符串，不会使原值或后续运算变成精确十进制计算。若问题要求精确货币规则，需选择适当数值表示和舍入协议，而不是任意增加显示位数。

2.5–2.8: 文本、边界与容器

2.5 固定输入可写

```
1 raw = " 82,91,0 "
2 values = [int(part.strip()) for part in raw.split(",")]
3 print(values)
```

结果 [82, 91, 0]。split 产生字符串列表，sum 的默认初值是数值 0，不能将 0 与字符串相加；即使文本拼接成功，也不等于数值求和。本题固定输入假定各项都是合法整数；任意输入应检查空项、数字格式和成绩范围，记录拒绝原因。

2.6 先排除非法范围，再从高阈值向低阈值判断：

```
1 def grade(score):
2     if not 0 <= score <= 100:
3         return None
4     if score >= 90:
5         return "A"
6     if score >= 60:
7         return "B"
8     return "C"
```

八个核对结果为 None、C、C、B、B、A、A、None。函数中的 return 会立即结束，故两个独立 if 在这里可正确工作；若改成多次赋值且不提前返回，就要重新检查覆盖问题。

2.7 结果分别为 [1, 3, 5]、[4, 5, 6]、[6, 5, 4, 3, 2, 1, 0] 和空列表。v[4:2] 默认步长为正，从 4 不能在小于 2 的边界内前进，故为空。v[:20] 截取到可用末尾并返回完整列表，v[20] 单项访问越界会报错。

2.8 append([3, 4]) 把二项列表当成第三个元素，结果为 [1, 2, [3, 4]]；extend([3, 4]) 逐项加入，结果为 [1, 2, 3, 4]。两种方法均修改原列表并返回 None，不能把返回值误当新列表。空字典为 {}，空集合为 set()。题中用列表字面量调用方法后，如果没有保存这个列表，就不能通过名称继续读取它，因此实际追踪时应先绑定变量。

2.9–2.12: 循环、映射与函数

2.9

```
1 def integer_sum(n):
2     if n < 0:
3         raise ValueError("n must be nonnegative")
4     total = 0
5     for i in range(1, n + 1):
6         total += i
7     return total
```

输入 0 时范围为空，返回 0；输入 1 返回 1；输入 5 返回 15。range 不含终点，要

包含 n 必须将终点设为 $n + 1$ 。本函数假定 n 为整数，若接收任意对象，还需另外检查类型。

2.10

```
1 x = 1
2 steps = 0
3 limit = 20
4 while x < limit:
5     x = 2 * x + 1
6     steps += 1
7 print(x, steps)
```

每轮结束 x 为 3、7、15、31，最终 4 轮。上界为 1 时，初始 $x < \text{limit}$ 就是假，不执行，输出 1 0。停止条件应在循环入口核对，不能先假定至少执行一次。

2.11

```
1 labels = ["cat", "dog", "cat", "bird", "dog", "cat"]
2 counts = {}
3 for label in labels:
4     counts[label] = counts.get(label, 0) + 1
5 unique = set(labels)
6 print(counts)
7 print(sorted(unique))
```

频数 cat 为 3、dog 为 2、bird 为 1，不同标签排序显示为 ['bird', 'cat', 'dog']。get 为未出现标签提供初始 0；此处 0 确实是合理频数。集合只保留不同元素，不记录同一标签第几次出现，因而不能替代原标签序列与特征配对。

2.12

```
1 def classify(score, threshold=60):
2     if not 0 <= score <= 100:
3         return None
4     if score >= threshold:
5         return "pass"
6     return "fail"
```

两个调用依次返回 "pass" 和 "fail"。这里还假定阈值为合法数值，若作为外部配置接收，可增加阈值范围检查。return 为调用者提供可继续使用的对象；print 只显示，并不把显示内容作为返回值交给调用者。

2.13–2.16: 计算与综合程序

2.13 两个函数可以共用输入核对：

```

1 def check_pairs(actual, predicted):
2     if len(actual) != len(predicted) or not actual:
3         raise ValueError("equal nonempty inputs required")
4
5
6 def mae(actual, predicted):
7     check_pairs(actual, predicted)
8     return sum(abs(p - y) for y, p in zip(actual, predicted)) / len(actual)
9
10
11 def mse(actual, predicted):
12     check_pairs(actual, predicted)
13     return sum((p - y) ** 2 for y, p in zip(actual, predicted)) / len(actual)

```

残差为 (1, -2, 0), MAE 为 1, MSE 为 5/3。分别以 ([], []) 和 ([1], [1, 2]) 调用应产生 `ValueError`, 而不是返回一个误导性的数。可与实验 2.2 相同的错误捕获方式核对; 输入列表应在调用前后保持相同内容。

2.14

```

1 values = [-2, 0, 3, -1, 4]
2 squares = [v ** 2 for v in values if v >= 0]
3 marks = ["valid" if v >= 0 else "invalid" for v in values]
4 print(squares)
5 print(marks)

```

平方结果为 [0, 9, 16], 长度 3。标记结果依次为 invalid、valid、valid、invalid、valid, 长度 5。第一种筛选删除不满足条件的项, 第二种对每个原始项都产生一个结果。

2.15 完整可运行参考答案还保存在 `ch02_solutions.py` 中。核心函数如下:

```

1 def clean_scores(raw):
2     clean = []
3     rejected = []
4     for index, text in enumerate(raw):
5         stripped = text.strip()
6         if not (stripped.isascii() and stripped.isdecimal()):
7             rejected.append((index, text, "invalid text"))
8             continue
9         value = int(stripped)
10        if not 0 <= value <= 100:
11            rejected.append((index, text, "out of range"))
12            continue
13        clean.append(value)
14    return clean, rejected

```

调用端先解包结果, 再检查是否有有效值, 最后决定是否计算均值:


```
1 values, rejected = clean_scores(["82", "", "0"])
2 if not values:
3     print("no valid scores")
4 else:
5     print(len(values), sum(values) / len(values))
```

当前输入有效值为 [82, 0]，数量 2，平均 41.0，拒绝下标 1。若输入两项均无效，则输出规定文本。函数新建结果列表，没有修改 `raw`；可保存调用前副本并比较调用后原列表，作为有限示例核对。

2.16

```
1 def nearest_one(train, x):
2     if not train:
3         raise ValueError("training data are empty")
4     ordered = sorted(train, key=lambda row: abs(row[0] - x))
5     return ordered[0][1]
6
7 train = [(1, 5), (2, 7), (4, 11)]
8 evaluation = [(1.2, 5.5), (3.5, 10)]
9 errors = []
10 for x, y in evaluation:
11     prediction = nearest_one(train, x)
12     errors.append(abs(prediction - y))
13     print(x, prediction)
14 print(sum(errors) / len(errors))
```

对 1.2，距离 0.2、0.8、2.8，选 (1,5)，预测 5，绝对误差 0.5。对 3.5，距离 2.5、1.5、0.5，选 (4,11)，预测 11，绝对误差 1。平均 0.75。预测函数只接收训练记录与新输入，未读取评价标签；评价列表在本题中已保证非空，通用评价函数仍需加非空检查。距离同时稳定排序保留原训练顺序。

2.19 本章小结

Python 脚本按控制流程执行，赋值绑定名称，表达式产生值，条件决定分支，循环逐项或按状态重复，函数用明确的输入输出封装计算。列表、元组、字典和集合对应不同的数据组织需求，不能只凭都能装多个元素而互相替换。

数值计算必须保持运算顺序、样本配对、分子分母和单位一致；浮点核对使用合理容差；无效输入和空数据需要明确处理。完成本章后，第一章的教学实验不再只是运行入口，而应成为可以逐行解释、手算核对和有控制地修改的程序。

资料说明与配套实验索引

本累计版只包含第 1 章《人工智能与机器学习全景》和第 2 章《Python 语言基础》。后续 8 篇 44 章的整体规划保存在同目录大纲中，未完成章节不列入本 PDF 正文目录。两章的编号、术语与实验编号供后续章节沿用。

本讲义中的例题、任务卡、教学数据与代码为针对本书学习目标编写的示例。基础语法可对照 Python 官方教程核对，可靠评价的流程可对照机器学习工具的官方实践说明。参考入口如下，访问日期为 2026 年 10 月 8 日；网页版本可能随官方维护更新，本章示例采用常见 Python 3 语法。

1. Python 官方教程：基础表达式与字符串。
<https://docs.python.org/3/tutorial/introduction.html>
2. Python 官方教程：控制流程与函数。
<https://docs.python.org/3/tutorial/controlflow.html>
3. Python 官方教程：数据结构。
<https://docs.python.org/3/tutorial/datastructures.html>
4. Python 官方教程：虚拟环境与包。
<https://docs.python.org/3/tutorial/venv.html>
5. Python 官方教程：浮点运算及其限制。
<https://docs.python.org/3/tutorial/floatingpoint.html>
6. scikit-learn 官方文档：常见问题与实践建议，特别是数据泄漏和预处理的一致性。
https://scikit-learn.org/stable/common_pitfalls.html

上列资料为相关机制的核对与进阶入口，不要求先通读才能运行本批实验。第一章的近邻、风险与评价指标将在相应后续章节补充完整算法、统计假设和推导。

入口文件	对应位置	内容
ch01_regression.py	实验 1.1	基线、近邻选择、最终测试
ch01_classification.py	实验 1.2	阈值、四类计数与指标
ch02_basics.py	实验 2.1	文本清理、拒绝记录、统计
ch02_metrics.py	实验 2.2	MAE、MSE、RMSE 与错误输入
ch02_solutions.py	习题 2.5–2.16	可运行的编程答案与核对
README.md	实验说明	环境、运行、预期与修改任务

实验输出记录另保存在“配套实验/预期输出”目录。主源文件为“源文件/机器学习讲义.tex”，可使用 XeLaTeX 继续编译与增补。源码集中定义版式与颜色，图示在文内生成，不需要另附图片才能编译。